# Multi-layer perception and Convolutional Neural Networks

Instructor: Hao Wang

AI Thrust, information hub



# Recaps of last week



# Linear Classifier



# A more typical framework

## K Nearest Neighbor

```
def train(images, labels):
  # Machine learning!
  return model
```



Memorize all data and labels

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the label of the most similar training image



# A more typical framework

## Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```



Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve $W^*$ that leads to minimal $L$

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the labels based on $f_{W^*}(x)$



# Regularization

$L_2$ regularization: $R(W) = \left|\left|W\right|\right|_2 = \sum_k W_k^2$

$L_1$ regularization: $R(W) = \left|\left|W\right|\right|_1 = \sum_k |W_k|$

**1. The "Maximum Margin" Principle**
In an SVM, the goal is to find a decision boundary that separates classes.
**Without Regularization:** The model might try too hard to correctly classify every single noise point in the training data, leading to a very complex boundary.
**With Regularization:** Minimizing $\left|\left|W\right|\right|_2$ is mathematically equivalent to **maximizing the margin.** A larger margin usually leads to better performance on new, unseen data because it's more robust.

2. Controlling the "Weights"
**Why small weights?** If a weight $W_i$ is very large, even a tiny change in a specific input feature $x_i$ will cause a huge swing in the output. By forcing the weights to be small, regularization ensures that the model doesn't over-rely on any single feature or specific noise in the training set. This leads to a **smoother** model.



# Loss function

Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$



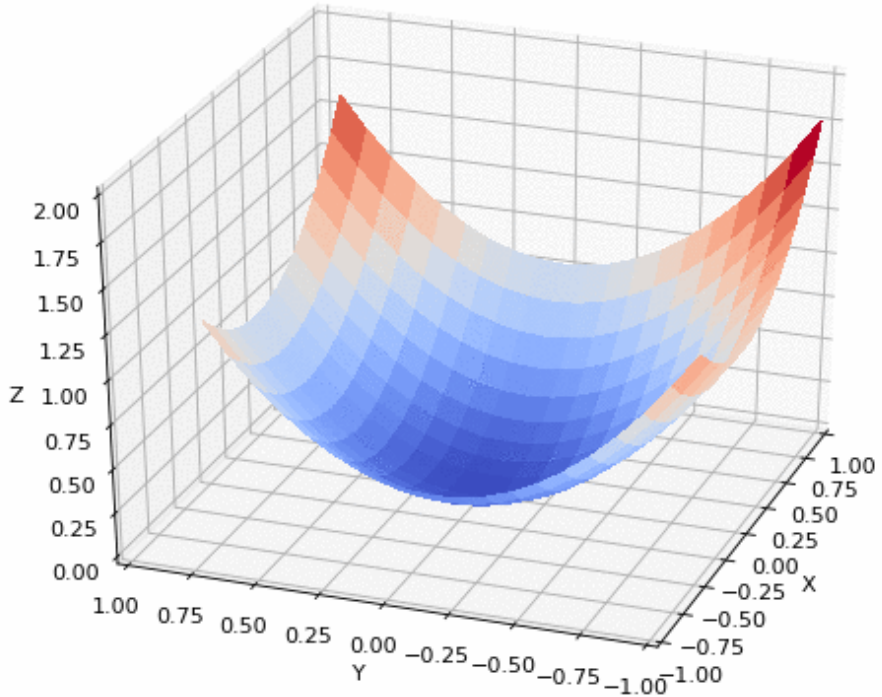
Probabilities must be >= 0

Probabilities must sum to 1

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| cat | **3.2** | exp | **24.5** | normalize | **0.13** | compare | **1** |
| car | 5.1 | | 164.0 | | 0.87 | | **0** |
| frog | -1.7 | | 0.18 | | 0.00 | | 0 |

$s = f(x_i, W)$ $\quad e^s$ $\quad \frac{e^{s_k}}{\sum_j e^{s_j}}$

# Gradient descent




Idea: follow the direction that leads to steepest loss reduction.

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W}$$

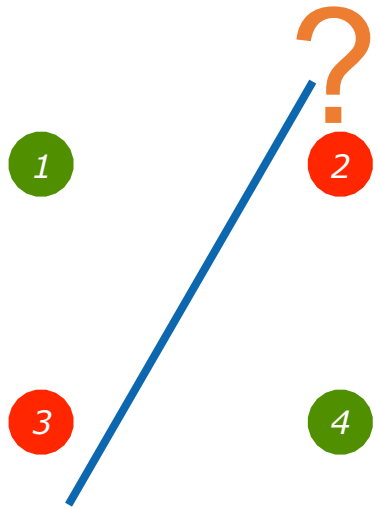
Step size

Gradient

# This week: Non-Linear Model

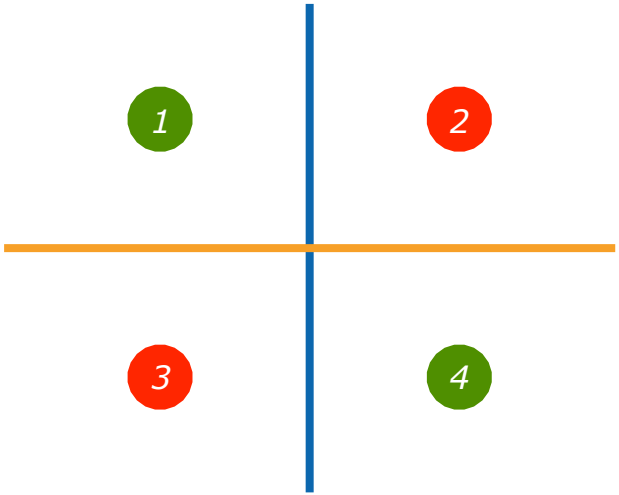


# Learning XOR

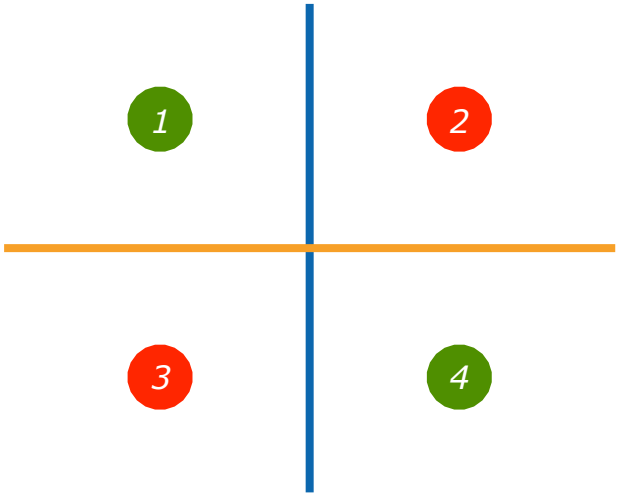



Linear classifier cannot handle XOR problem

# Learning XOR



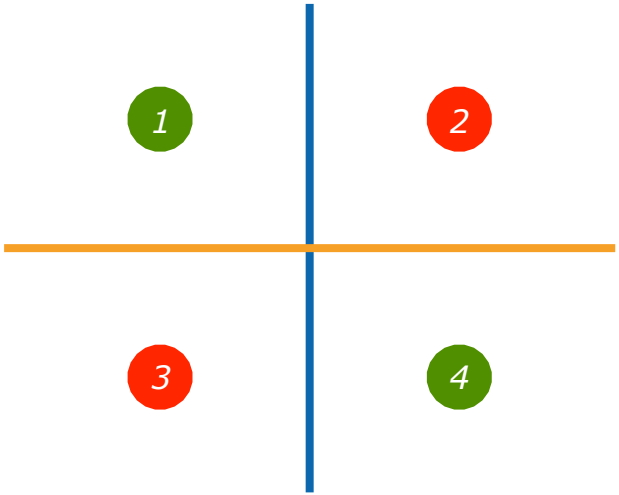
# Learning XOR



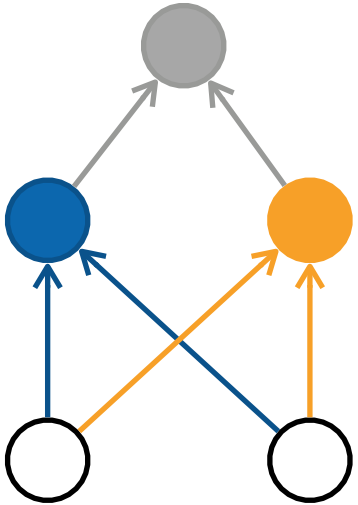

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| | + | - | + | - |
| | + | + | - | - |
| product | + | - | - | + |

*By combining the two linear features, we have successfully separated the green points (+) from the red points (-)!*

# Learning XOR




| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| | + | - | + | - |
| | + | + | - | - |
| product | + | - | - | + |

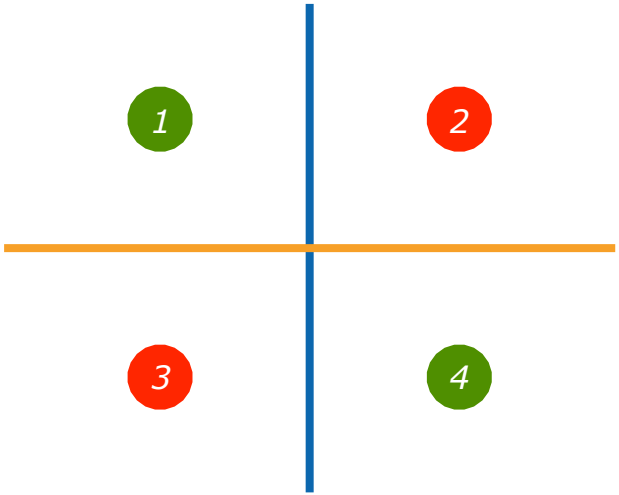


***Output Layer:*** *This node takes the outputs from the hidden layer and combines them to make the final classification (the last rows of the table).*

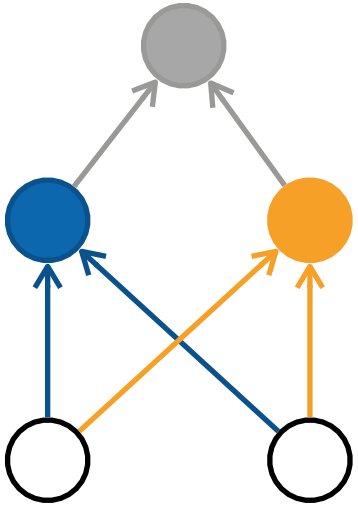
***Hidden Layer:*** *These nodes represent the two lines drawn on the graph. They take the raw coordinates and apply the linear transformations (the first two rows of the table).*

***Input Layer****: These represent the original x and y coordinates of the data points.*

# Learning XOR




| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| | + | - | + | - |
| | + | + | - | - |
| product | + | - | - | + |

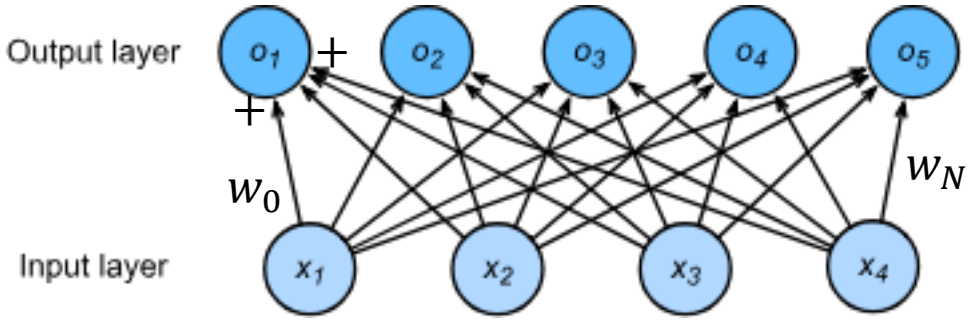


***Question:*** *Is this a linear model? Why?*

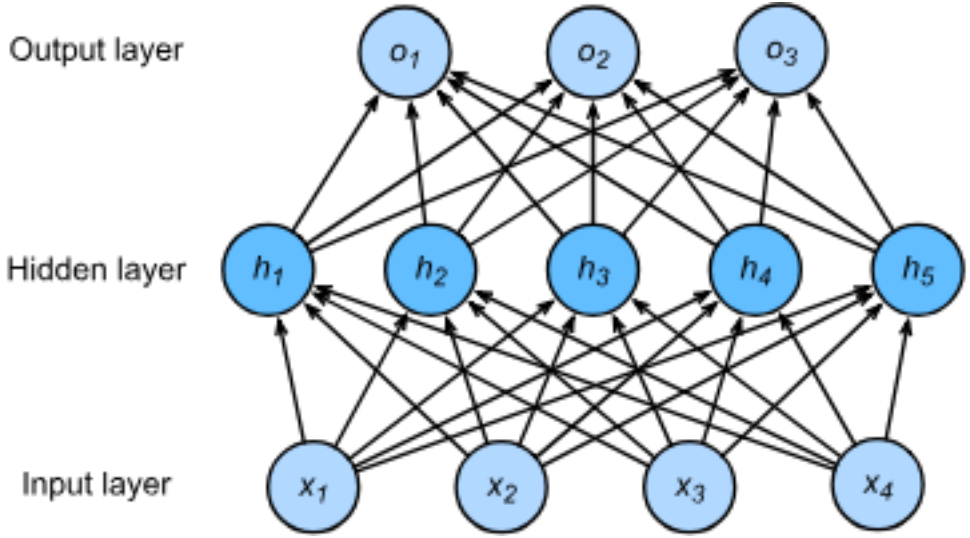
***Answer:*** *No!*

- **The Threshold** *(The Activation Function, Step Function)*
- **The Product Operation** *(Multiplying variables together, like $x_1 \times x_2$) or applying logical rules is a fundamentally non-linear operation.)*

# Single Hidden Layer



# Single Hidden Layer




Hyperparameter - size m of hidden layer

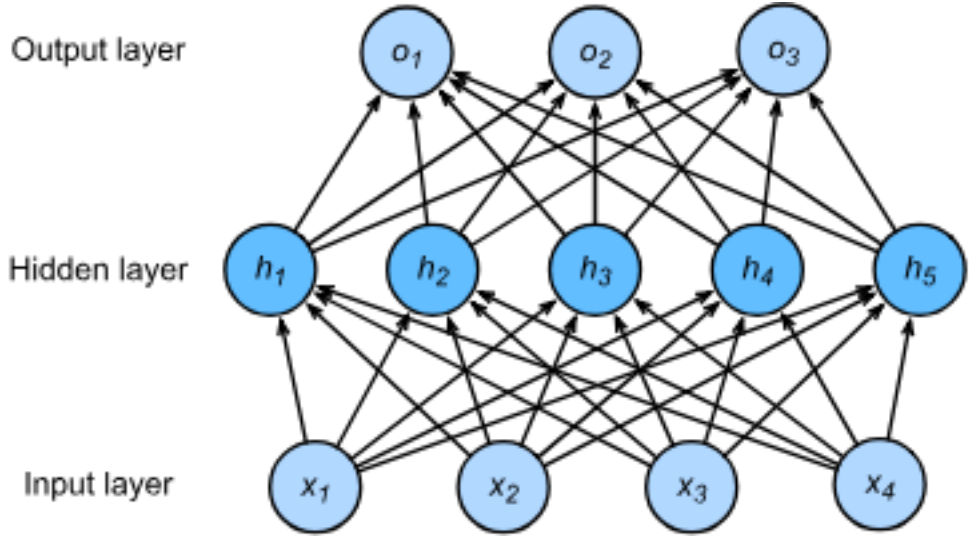
# Single Hidden Layer

- Input $\mathbf{x} \in \mathbb{R}^n$
- Hidden $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$, $\mathbf{b}_1 \in \mathbb{R}^m$
- Output $\mathbf{W}_2 \in \mathbb{R}^m$, $\mathbf{b}_2 \in \mathbb{R}$

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}_2^T\mathbf{h} + b_2$$

$\sigma$ is an element-wise activation function



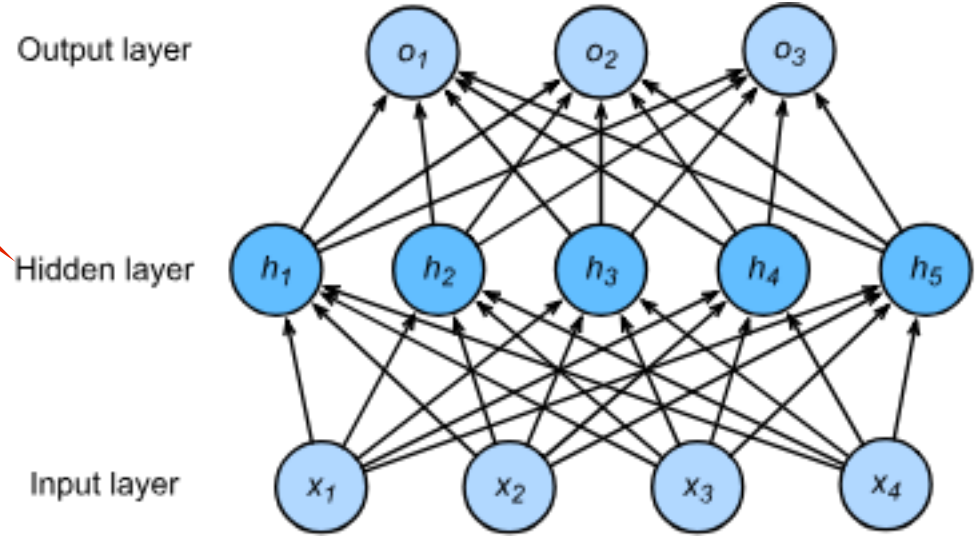
# Single Hidden Layer

Why do we need a nonlinear activation?

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}_2^T\mathbf{h} + b_2$$

$\sigma$ is an element-wise activation function






# Single Hidden Layer

A linear transformation of a linear transformation is still a linear transformation!

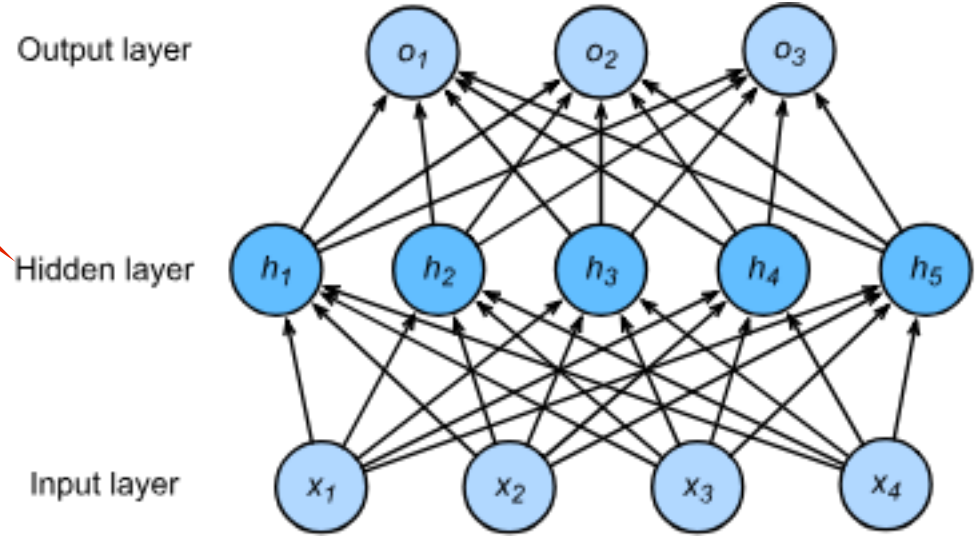
Why do we need a nonlinear activation?

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$$

$$\mathbf{o} = \mathbf{w}^T_2\mathbf{h} + b_2$$

hence $o = \mathbf{w}^\top_2\mathbf{W}_1\mathbf{x} + b'$

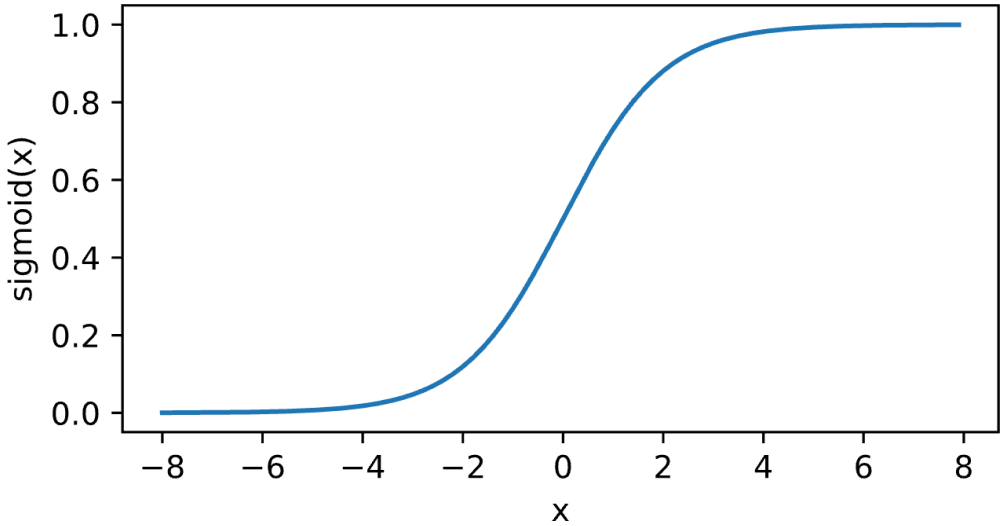
Linear …



# Sigmoid Activation

Map input into (0, 1), a soft version of $\sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$

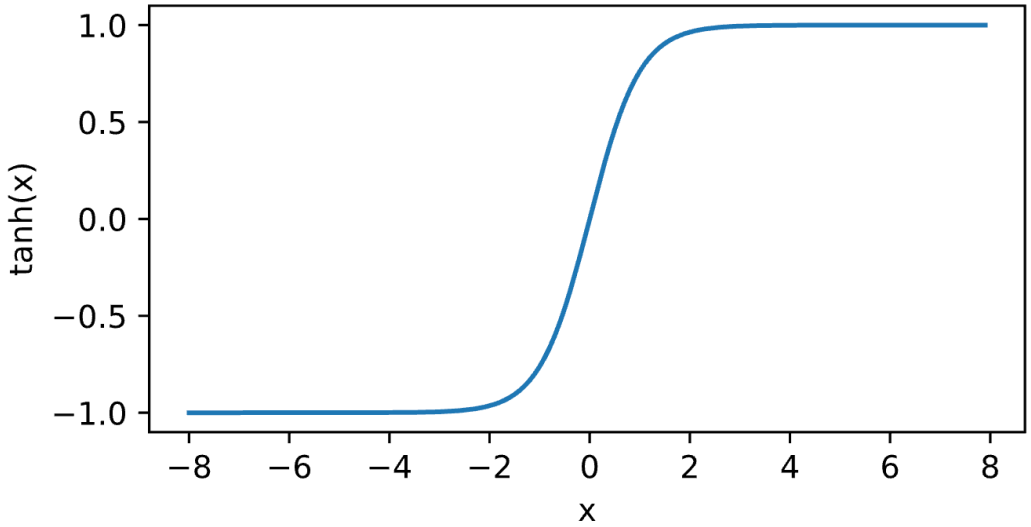
$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}$$



# Tanh Activation

Map inputs into (-1, 1)

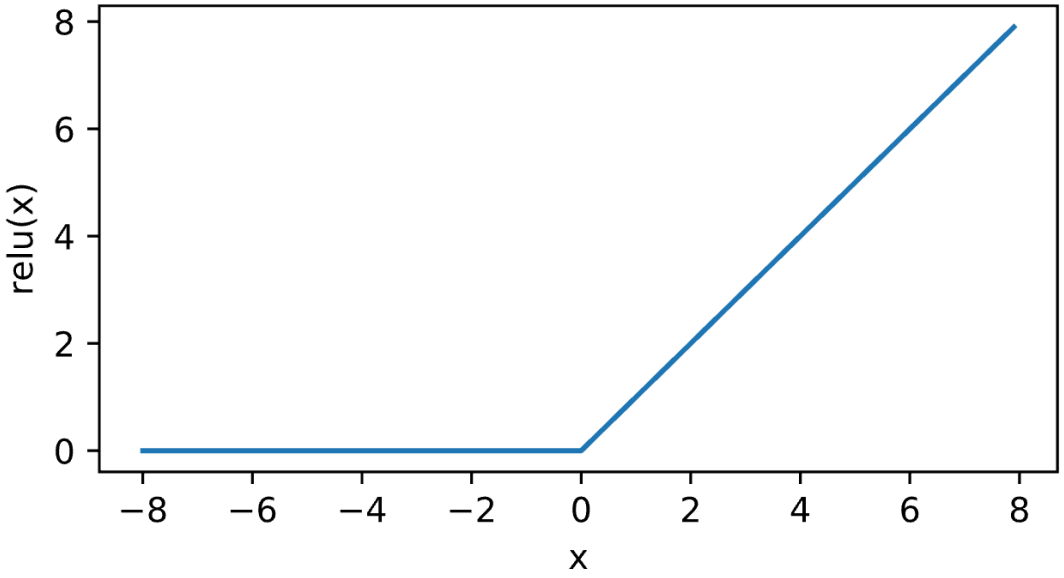
$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}$$



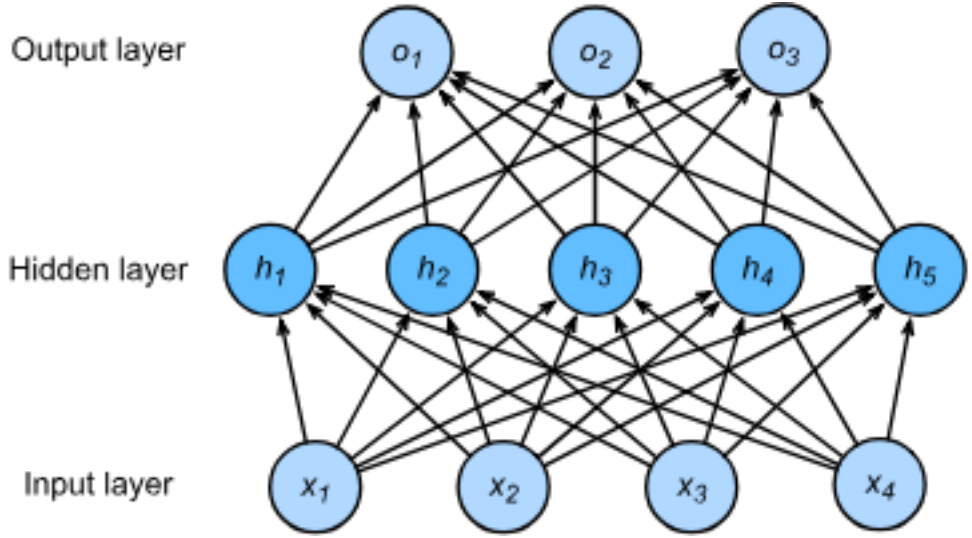



# ReLU Activation

ReLU: rectified linear unit: $\mathrm{ReLU}(x) = \max(x, 0)$

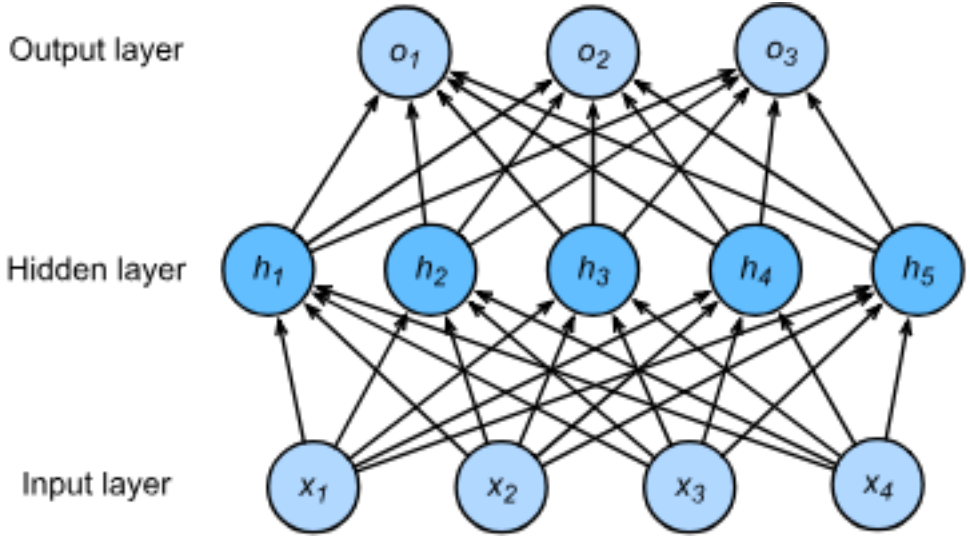


# Multiclass Classification



# Multiclass Classification

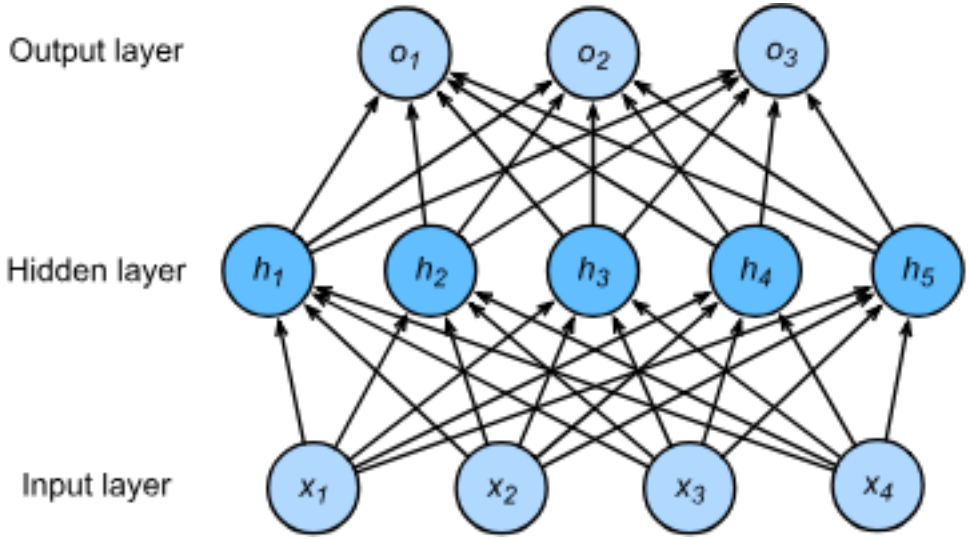
$$y_1, y_2, \ldots, y_k = \text{softmax}(o_1, o_2, \ldots, o_k)$$






# Multiclass Classification

- Input $\mathbf{x} \in \mathbb{R}^n$
- Hidden $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$ and $\mathbf{b}_1 \in \mathbb{R}^m$
- Output $\mathbf{W}_2 \in \mathbb{R}^{m \times d}$ and $\mathbf{b}_2 \in \mathbb{R}^d$



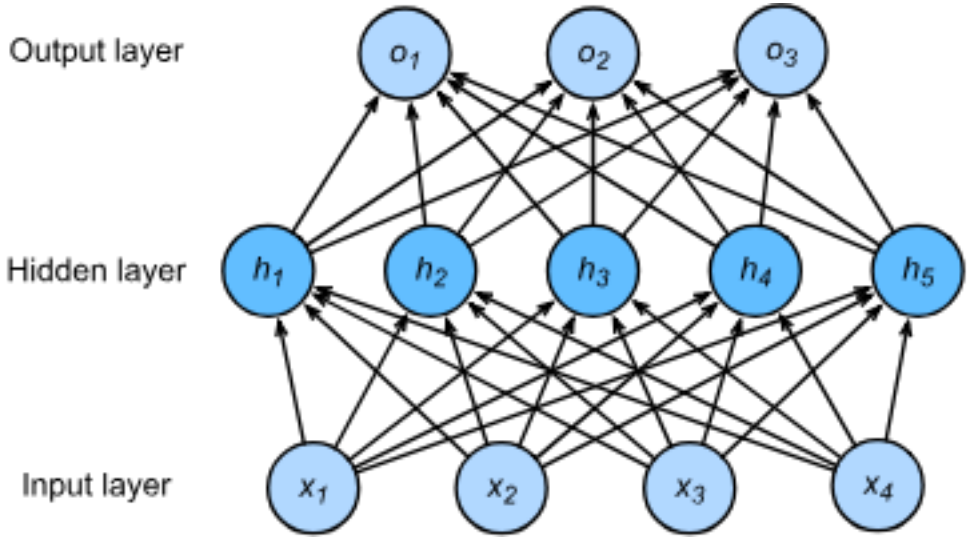
# Multiclass Classification

- Input $\mathbf{x} \in \mathbb{R}^n$
- Hidden $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$ and $\mathbf{b}_1 \in \mathbb{R}^m$
- Output $\mathbf{W}_2 \in \mathbb{R}^{m \times d}$ and $\mathbf{b}_2 \in \mathbb{R}^d$

$$\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}_2^T \mathbf{h} + \mathbf{b}_2$$

$$\mathbf{y} = \text{softmax}(o)$$

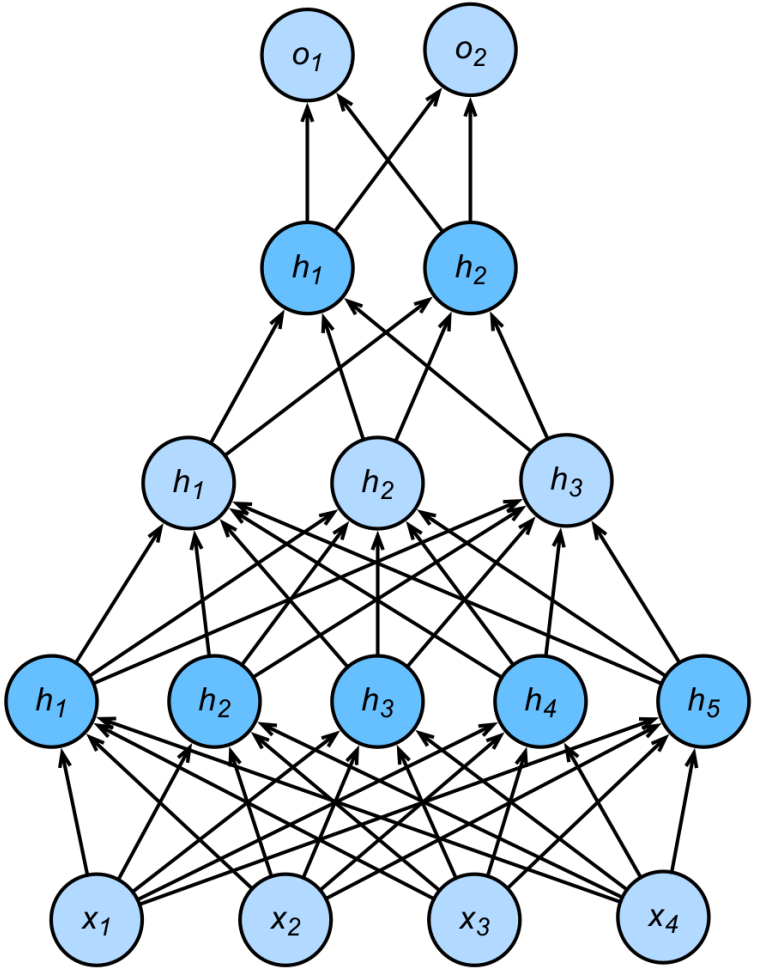


# Multiple Hidden Layers

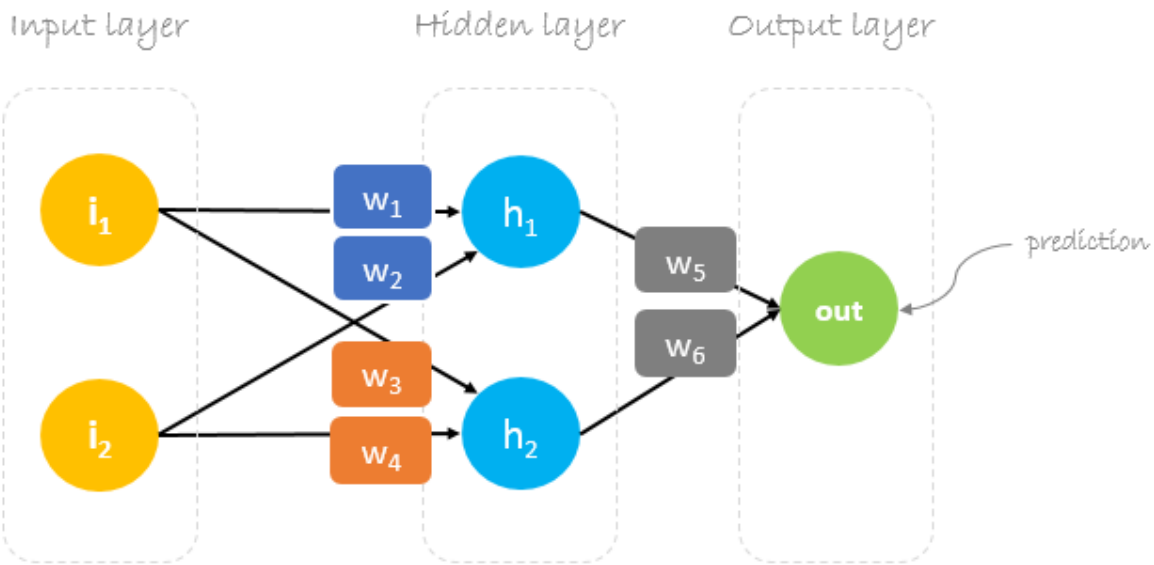
$$\mathbf{h}_1 = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$\mathbf{h}_2 = \sigma(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$$
$$\mathbf{h}_3 = \sigma(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3)$$
$$\mathbf{o} = \mathbf{W}_4\mathbf{h}_3 + \mathbf{b}_4$$

Hyper-parameters

- Number of hidden layers
- Hidden size for each layer

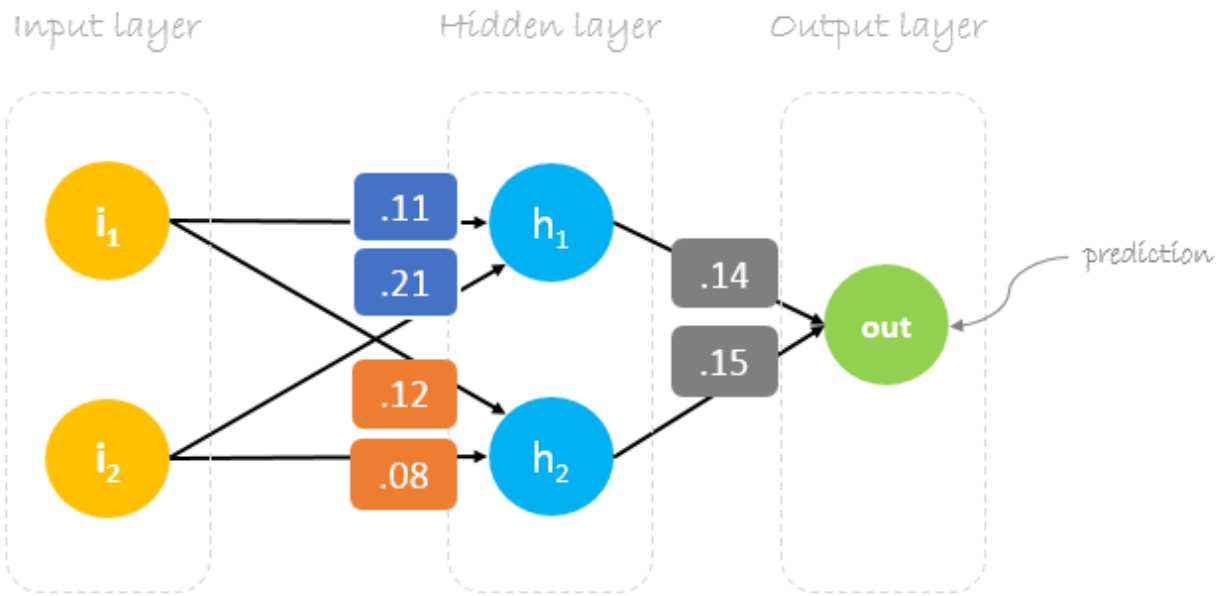


# Training MLP: Backpropagation



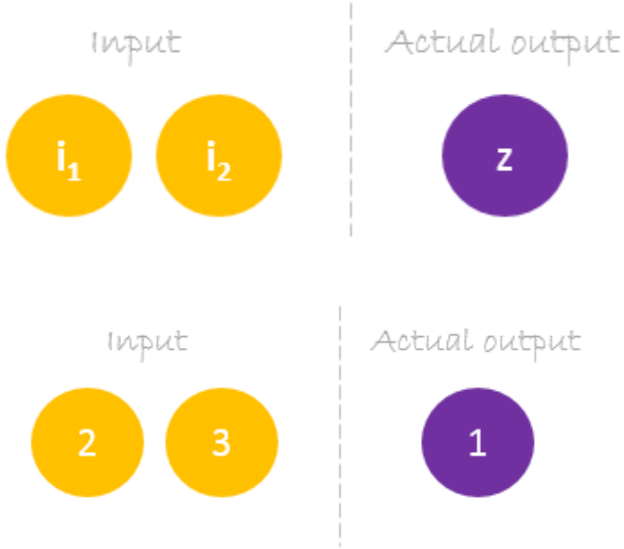

https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation



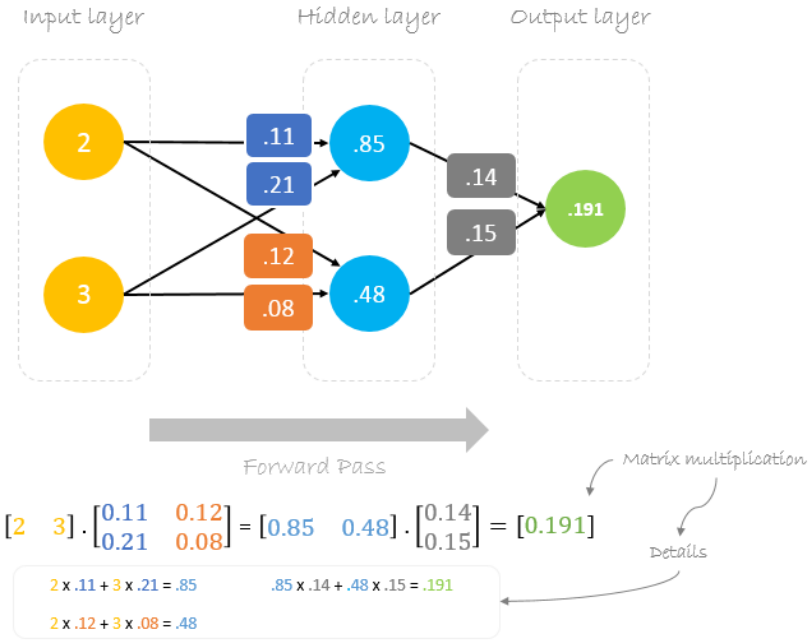

https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation



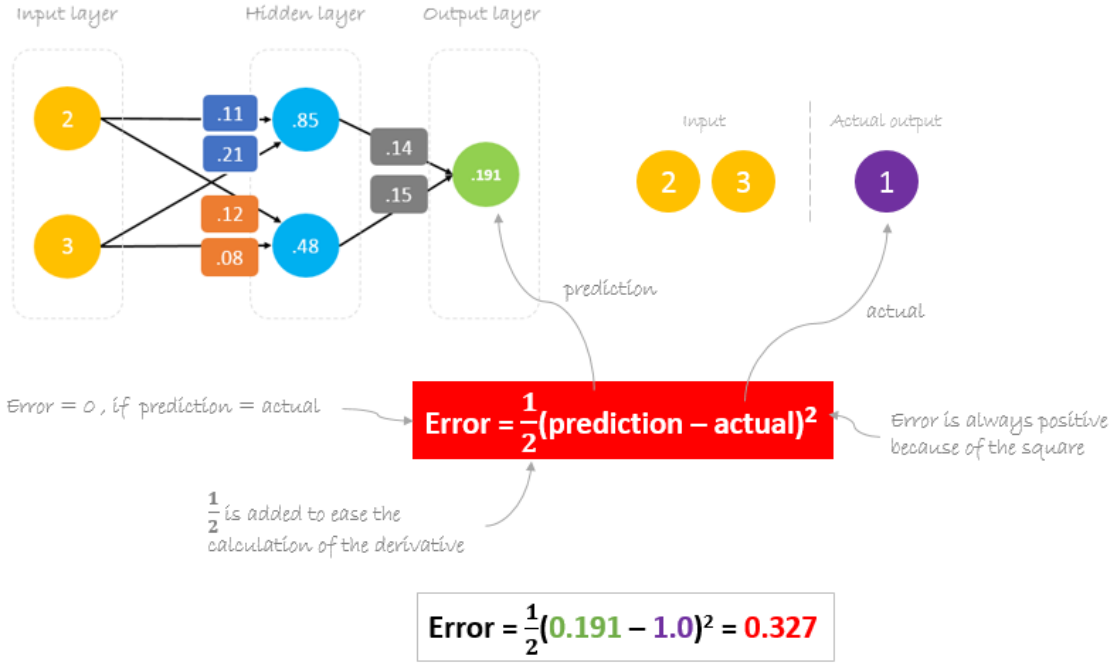

https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation




https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation




https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation

$$\text{prediction} = \text{out}$$

$$\text{prediction} = (h_1)\, w_5 + (h_2)\, w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

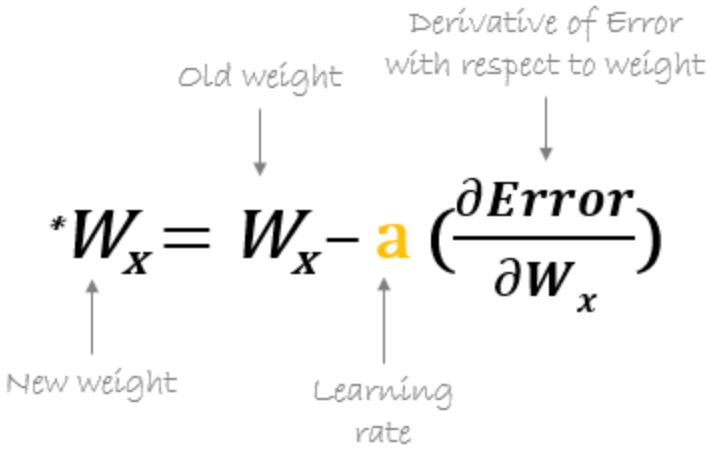
$$h_2 = i_1 w_3 + i_2 w_4$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2)\, w_5 + (i_1 w_3 + i_2 w_4)\, w_6$$

to change ***prediction*** value,
we need to change ***weights***

https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation




https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation

$$^{*}W_6 = W_6 - a\left(\frac{\partial Error}{\partial W_6}\right)$$



https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation

$$\frac{\partial Error}{\partial W_6} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial W_6}$$

chain rule

Error = $\frac{1}{2}$(prediction – actual)$^2$

prediction = $(i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$

$$\frac{\partial Error}{\partial W_6} = \frac{\frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial W_6}$$

$$\frac{\partial Error}{\partial W_6} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (i_1 w_3 + i_2 w_4)$$

$h_2 = i_1 w_3 + i_2 w_4$

$$\frac{\partial Error}{\partial W_6} = (\mathbf{predictoin - actula}) * (h_2)$$

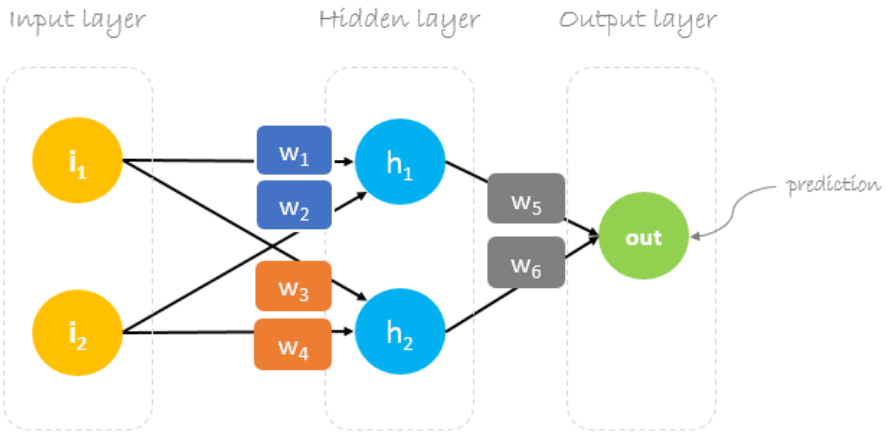
Δ = prediction – actual

delta

$$\frac{\partial Error}{\partial W_6} = \Delta h_2$$

https://hmkcode.com/ai/backpropagation-step-by-step/

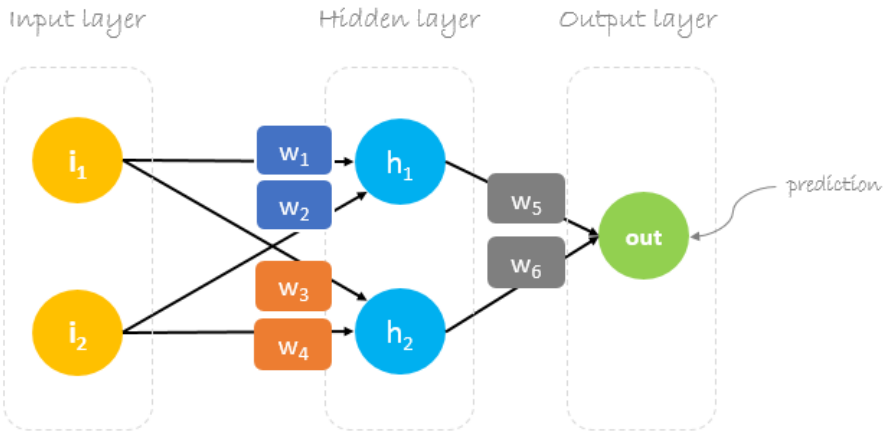
# Training MLP: Backpropagation




$$*W_6 = W_6 - a\,\Delta h_2$$

https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation




$$^{*}W_5 = W_5 - a\,\Delta h_1$$

https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation

$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$

chain rule

$$\frac{\partial Error}{\partial W_1} = \frac{\partial \frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (h_1)\, w_5 + (h_2)\, w_6}{\partial h_1} * \frac{\partial\, i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial Error}{\partial W_1} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (w_5) * (i_1)$$

$$\frac{\partial Error}{\partial W_1} = (\mathbf{predictoin - actula}) * (w_5 i_1)$$

$$\frac{\partial Error}{\partial W_1} = \Delta\, w_5 i_1$$

**Error** = $\frac{1}{2}$(prediction – actual)$^2$

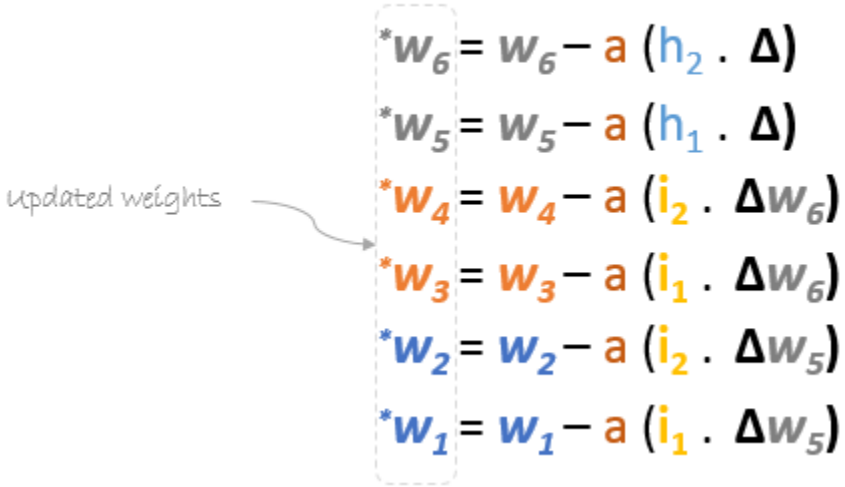
**prediction** = $(h_1)\, w_5 + (h_2)\, w_6$

$h_1 = i_1 w_1 + i_2 w_2$

**Δ = prediction – actual** ← delta

https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation




https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a\,\Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a\,\Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} . [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a\, i_1 \Delta w_5 & a\, i_1 \Delta w_6 \\ a\, i_2 \Delta w_5 & a\, i_2 \Delta w_6 \end{bmatrix}$$

https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation

$\Delta = 0.191 - 1 = -0.809$ ← **Delta** = prediction - actual

$a = 0.05$ ← **Learning rate,** we smartly guess this number

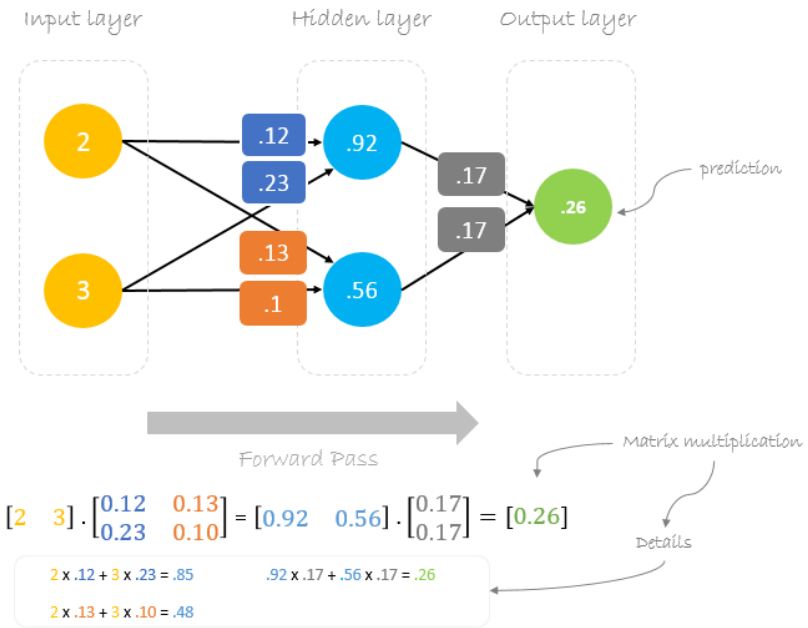
$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} . \begin{bmatrix} 0.14 & 0.15 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$



https://hmkcode.com/ai/backpropagation-step-by-step/

# Training MLP: Backpropagation




With the update weights, the prediction is 0.26 (previous 0.191), closer to the ground truth 1.

https://hmkcode.com/ai/backpropagation-step-by-step/

# Playground

- https://playground.tensorflow.org

- Use default setting to see the behavior
- Change Sigmoid to tanh
- Use sigmoid/Tanh, and 5 layers, to see the learning behavior
- Change it to ReLU.

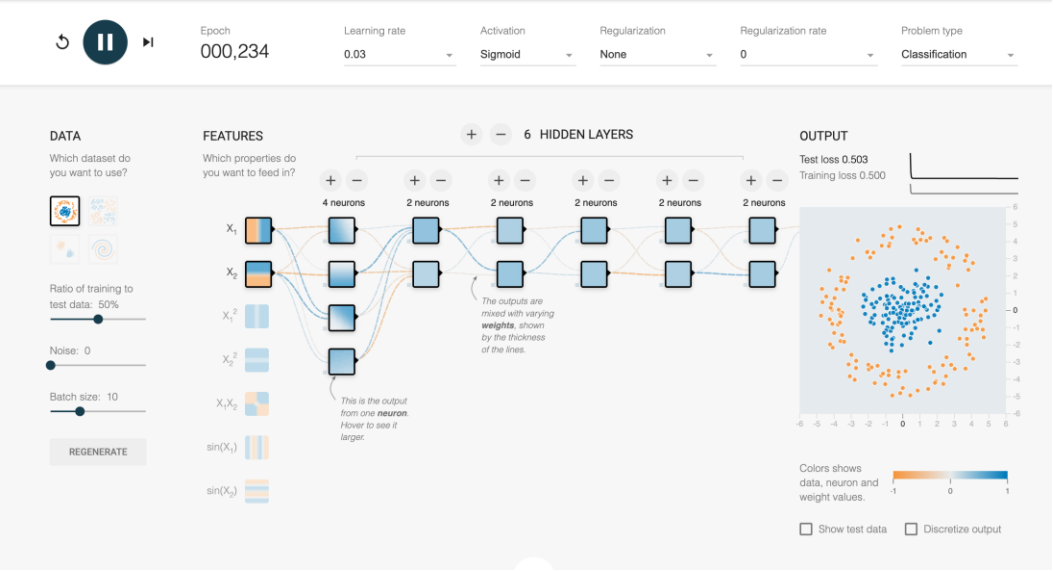


# Activation Function



# Some observations

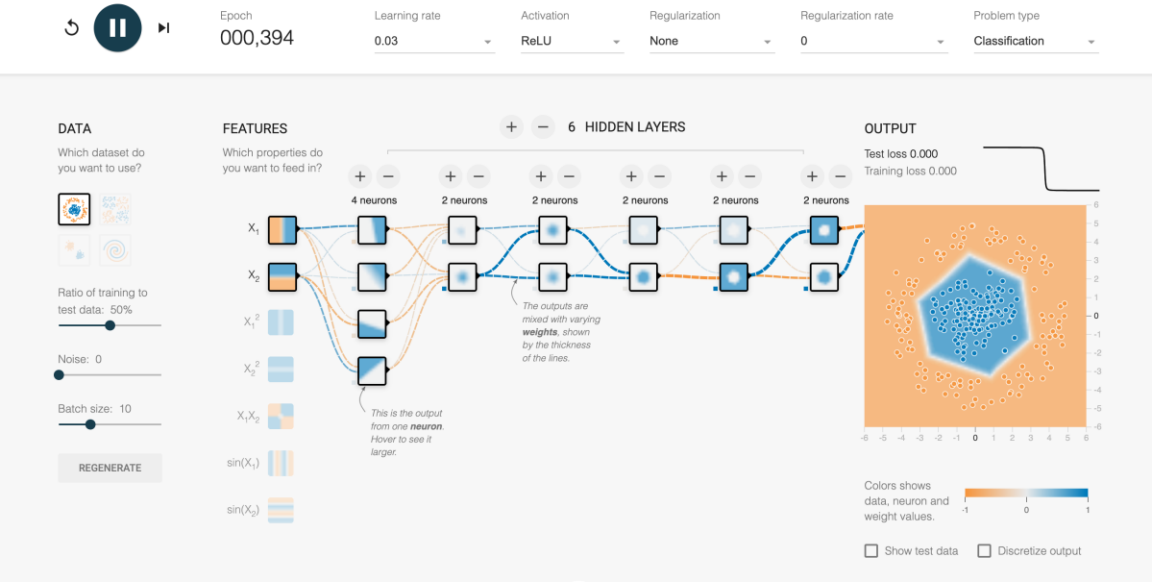
- The network don’t converge if we use a lot of layers




https://playground.tensorflow.org

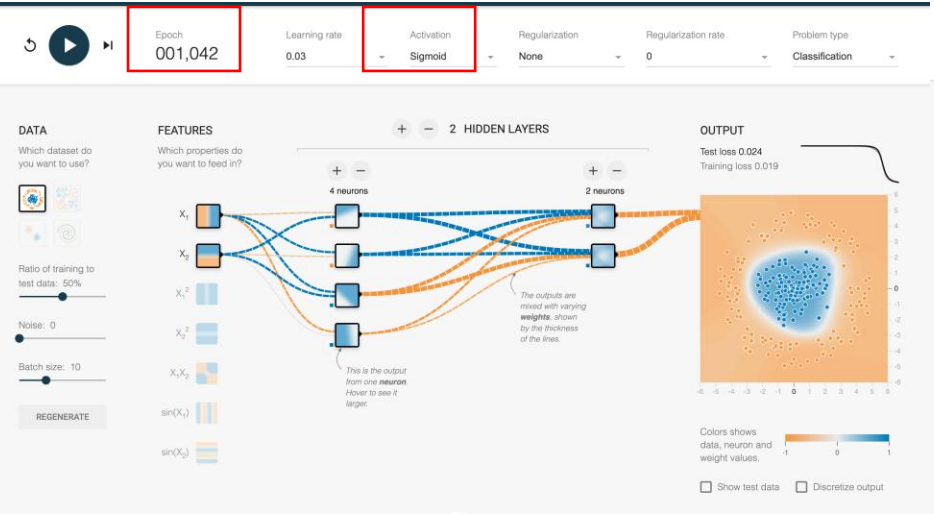
# Some observations

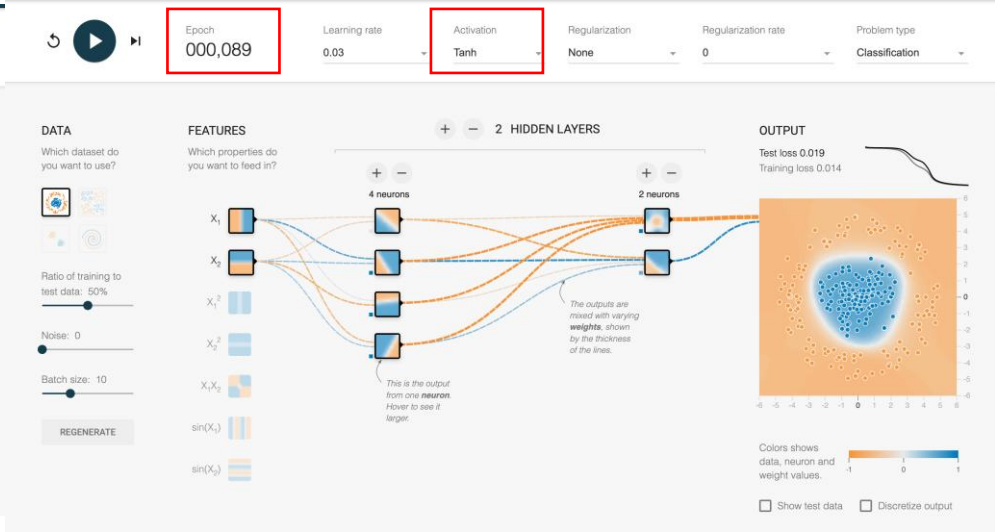
- Changing Sigmoid to ReLU solves this problem



# Some observations

- Tanh learns faster than sigmoid

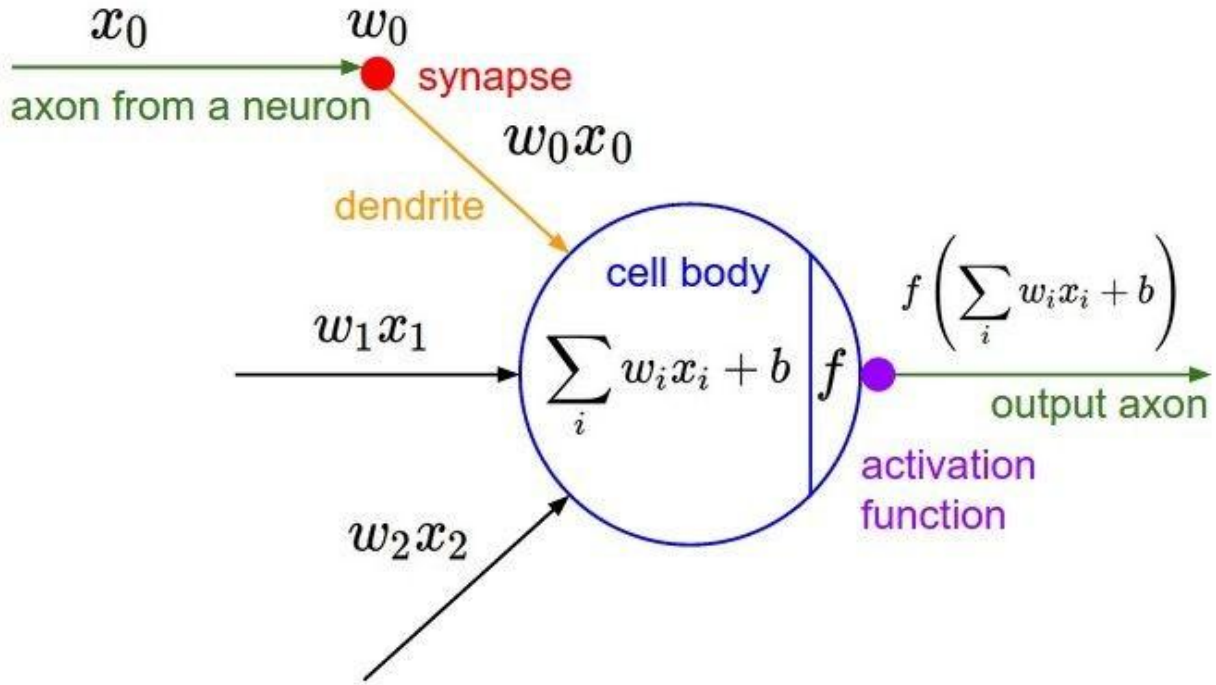


# Activation Functions



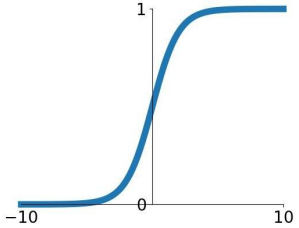
# Activation Functions



# Activation Functions

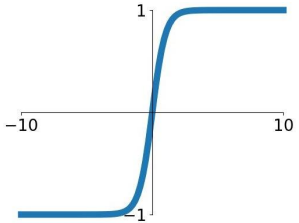
**Sigmoid**

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



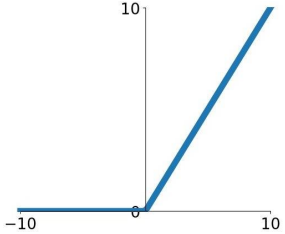

**tanh**

$$\tanh(x)$$



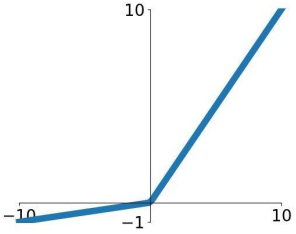

**ReLU**

$$\max(0, x)$$




**Leaky ReLU**

$$\max(0.1x, x)$$

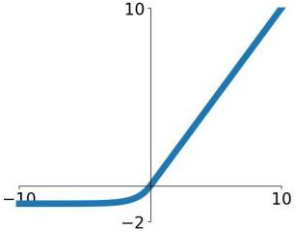



**Maxout**

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

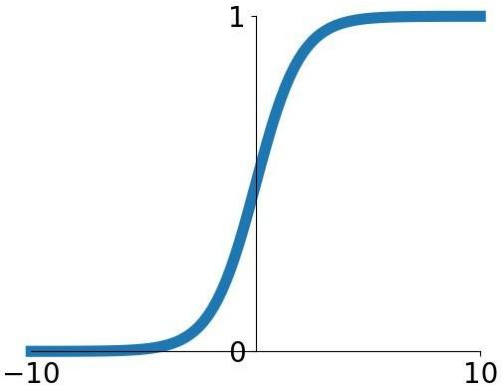
**ELU**

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



# Activation Functions

$$\sigma(x) = 1/(1 + e^{-x})$$



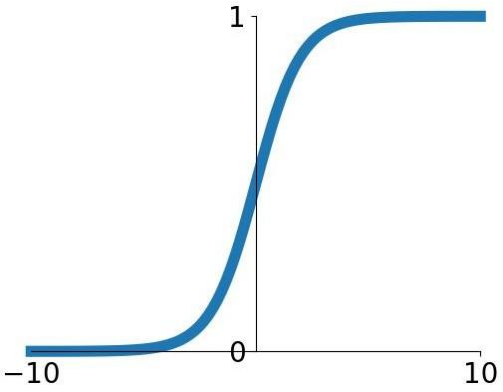

**Sigmoid**

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron



# Activation Functions

$$\sigma(x) = 1/(1 + e^{-x})$$

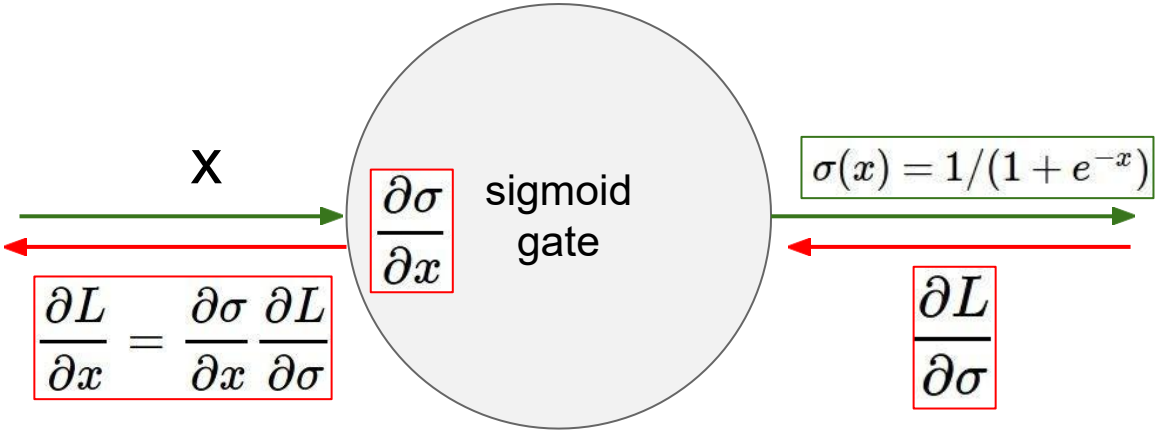


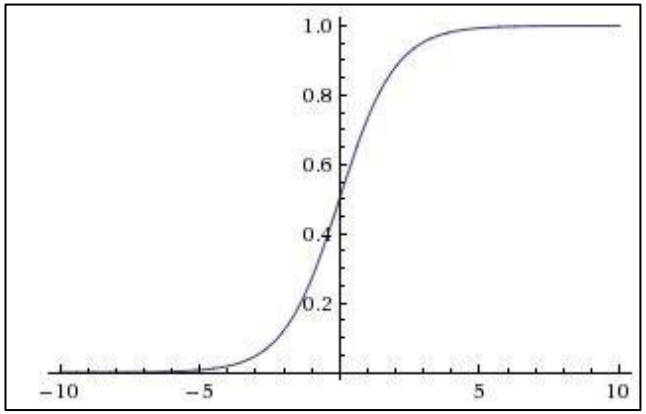

**Sigmoid**

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

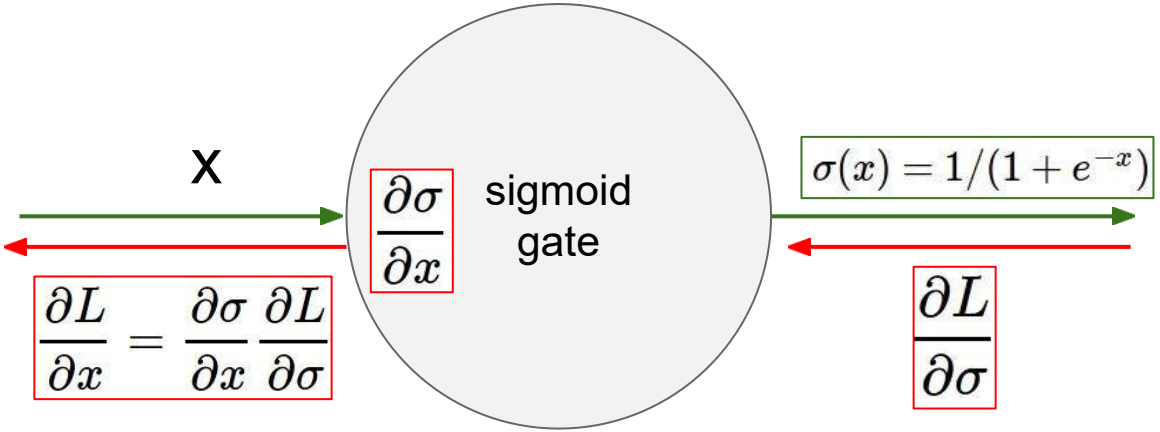
1. Saturated neurons “kill” the gradients

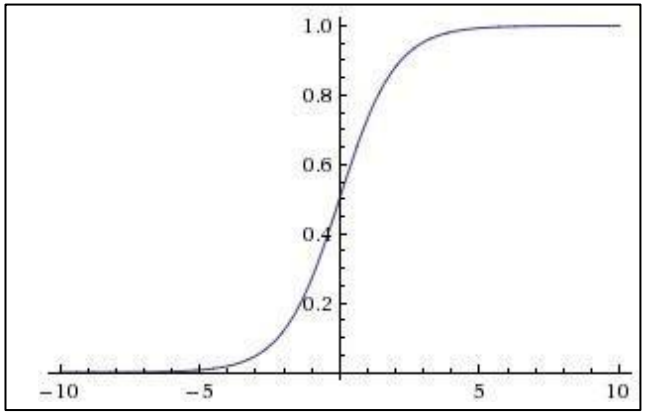




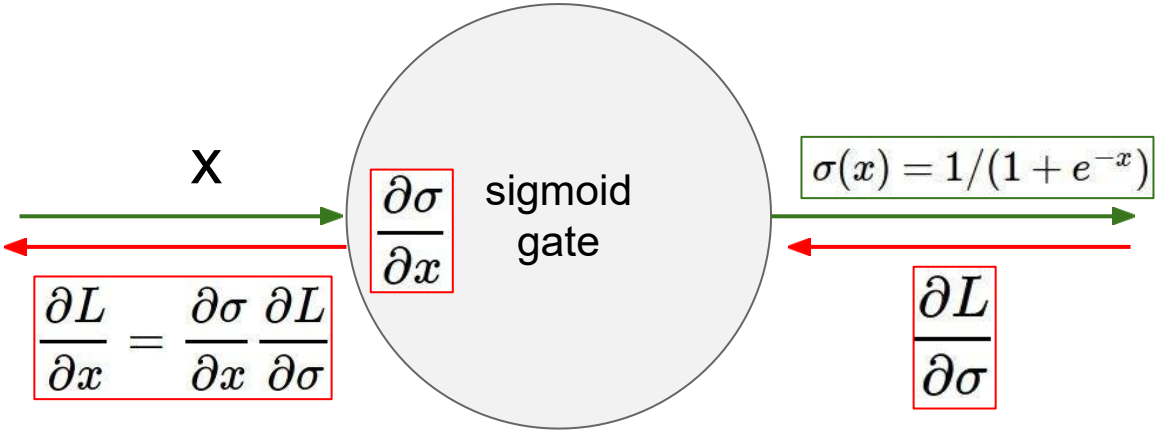

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

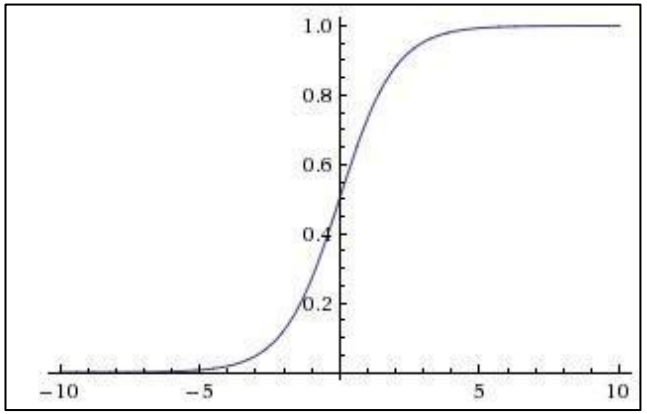



What happens when x = -10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$



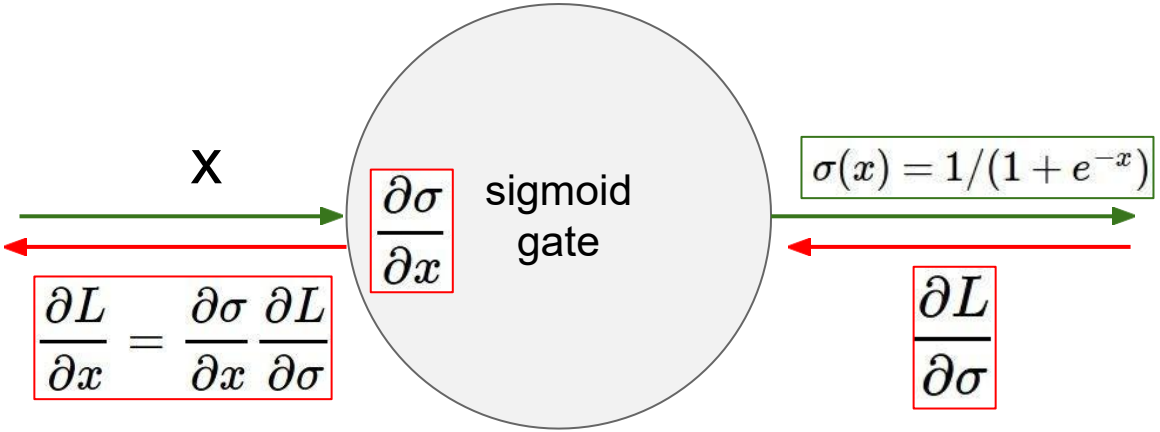



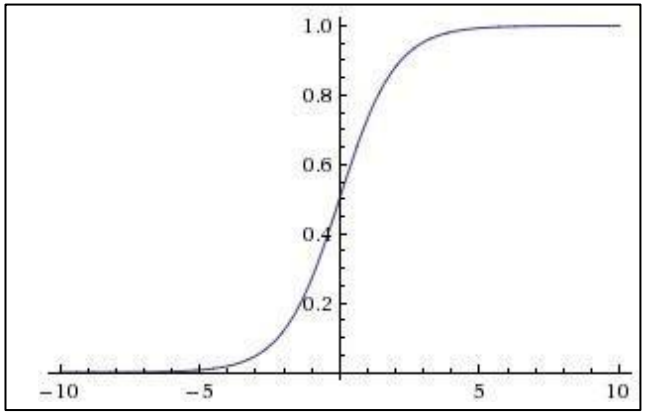

## What happens when x = -10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)\,)$$

$$\sigma(x) = \sim 0$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)\,) = 0(1-0) = 0$$

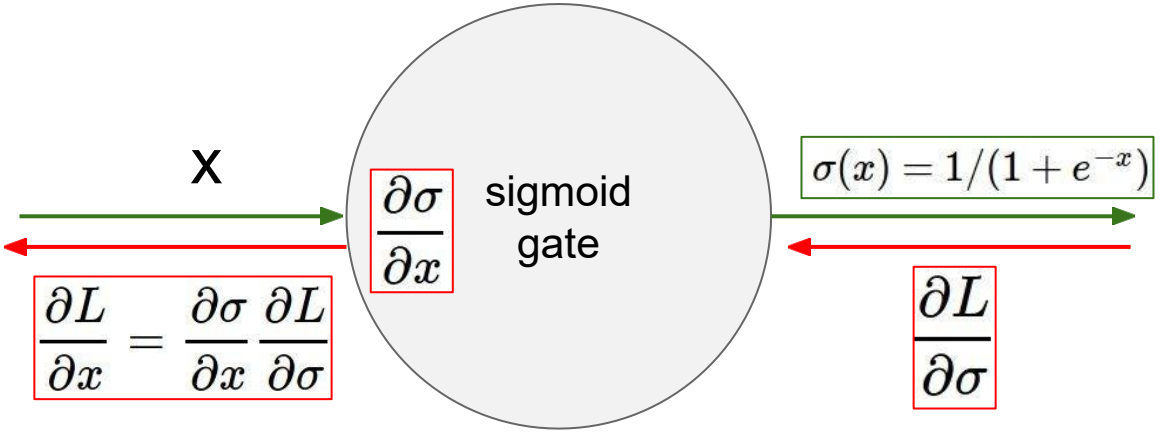


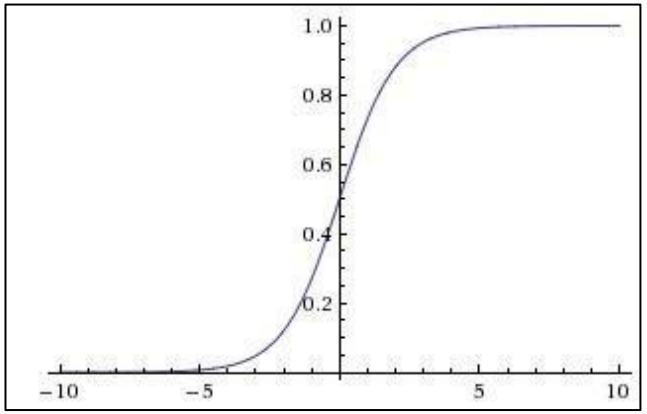




What happens when x = -10?

What happens when x = 0?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$



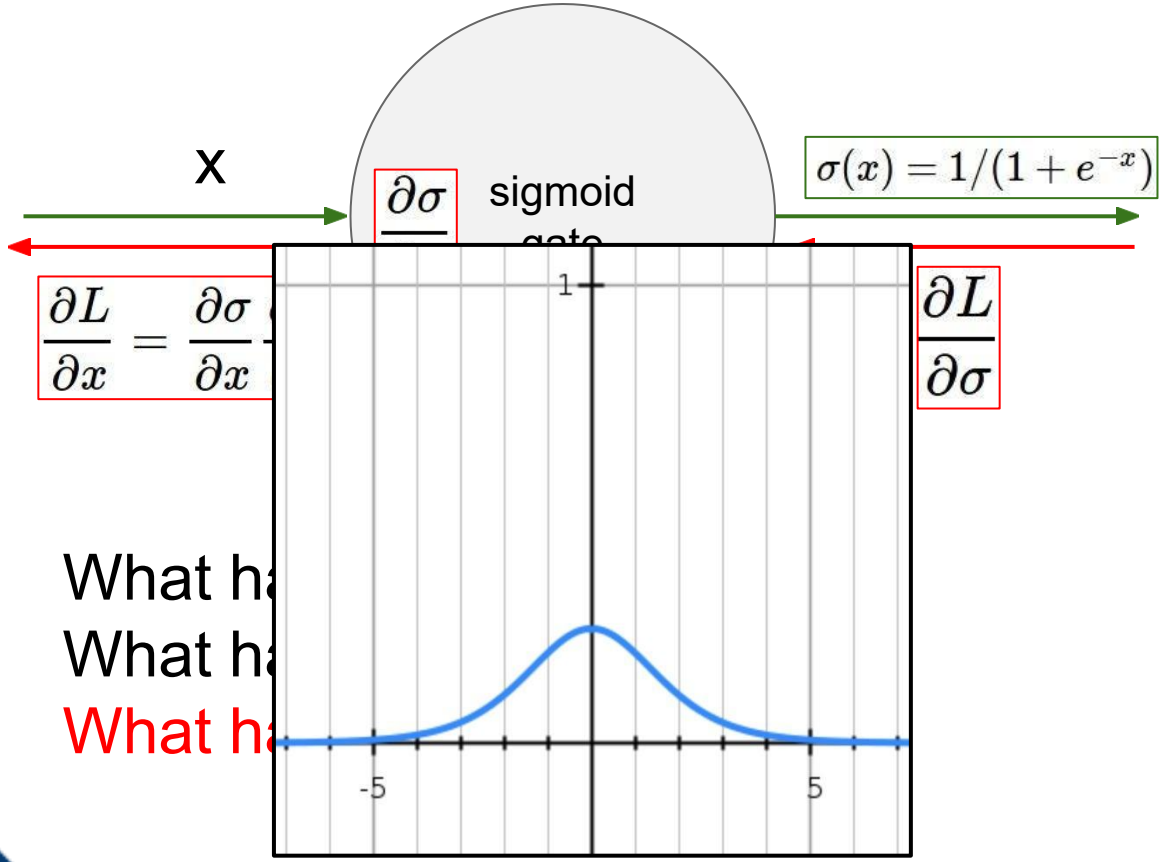

What happens when x = -10?

What happens when x = 0?

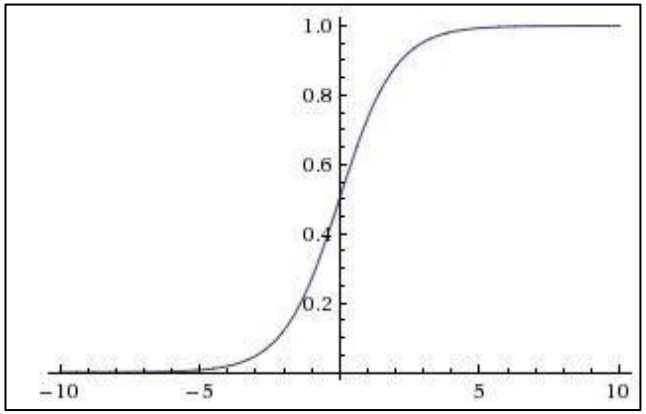
What happens when x = 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

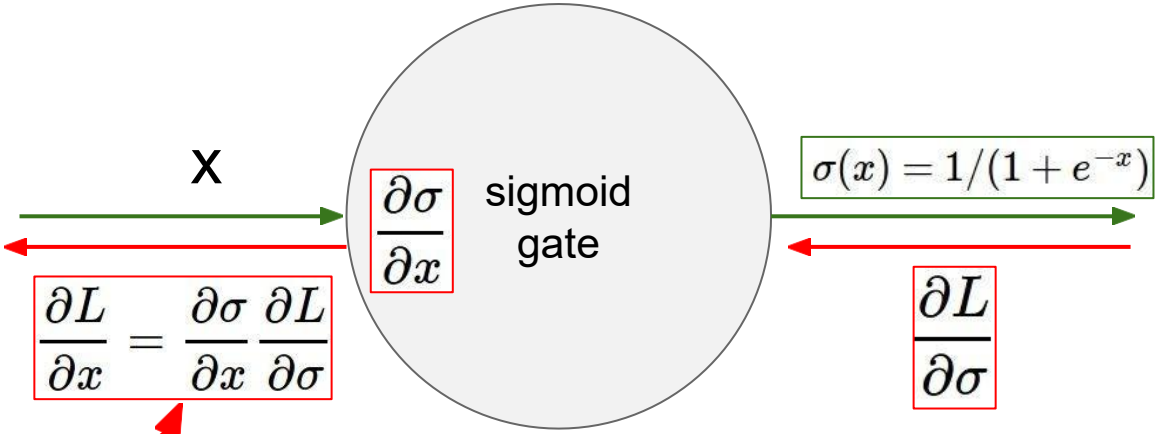
$$\sigma(x) = \sim 1 \qquad \frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)) = 1(1-1) = 0$$

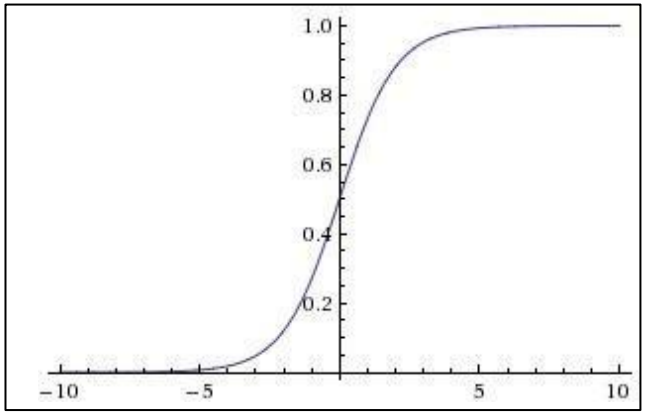



Gradient




$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$



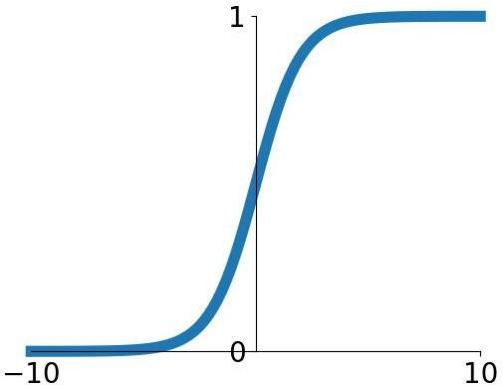




$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

Why is this a problem?

If all the gradients flowing back will be zero and weights will never change

# Activation Functions




**Sigmoid**

$$\sigma(x) = 1/(1+e^{-x})$$

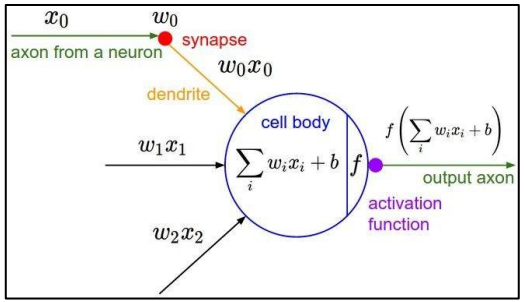
- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

1. Saturated neurons “kill” the gradients
2. Sigmoid outputs are not zero-centered ***(always positive)***

Why this is bad?

Consider what happens when the input to a neuron is always positive...

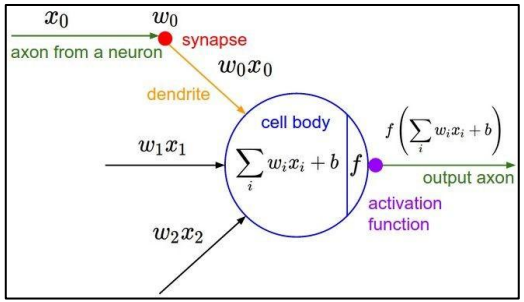
$$f\left(\sum_i w_i x_i + b\right)$$




What can we say about the gradients on **w**?

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



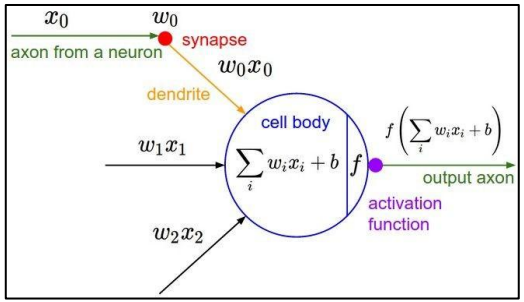

What can we say about the gradients on **w**?

$$\frac{\partial L}{\partial w} = \sigma(\textstyle\sum_i w_i x_i + b)(1 - \sigma(\textstyle\sum_i w_i x_i + b))x \times upstream_gradient$$



Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



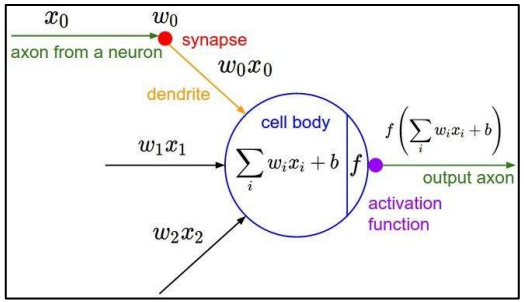

What can we say about the gradients on **w**?

We know that local gradient of sigmoid is always positive

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



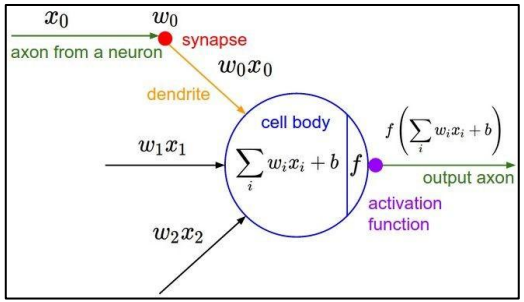

What can we say about the gradients on **w**?

We know that local gradient of sigmoid is always positive
We are assuming x is always positive

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right)x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$




What can we say about the gradients on **w**?

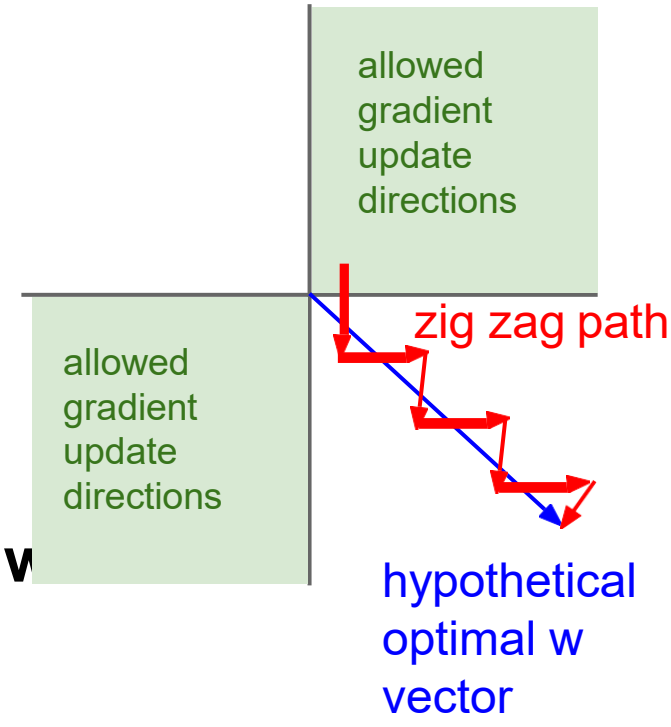
We know that local gradient of sigmoid is always positive
We are assuming x is always positive

So!! Sign of gradient **for all $w_i$** is the same as the sign of upstream scalar gradient!

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right)x \times upstream_gradient$$

# Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



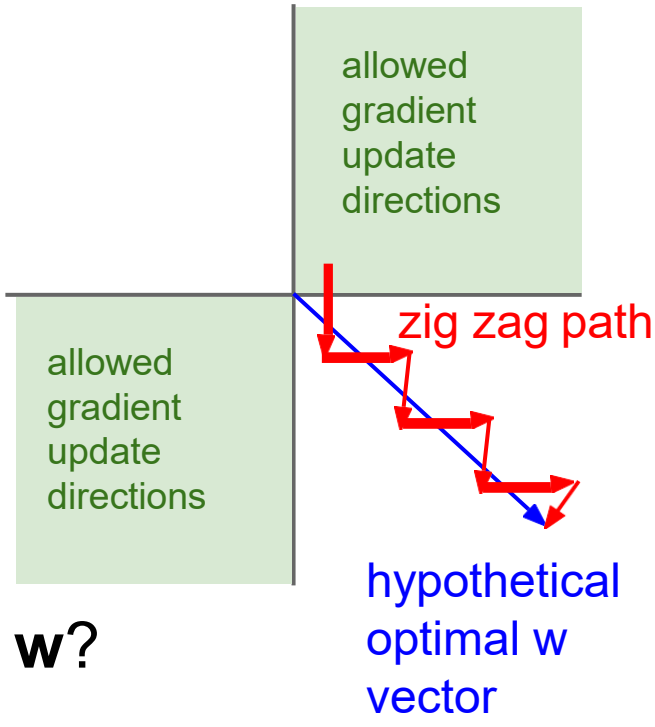

What can we say about the gradients on **w**

Always all positive or all negative

- Imagine a scenario where the optimal solution requires weight $w_1$ to increase, but weight $w_2$ to decrease.
- Because the inputs are not zero-centered, the network **cannot make these opposite adjustments at the same time**. It is forced to either increase all weights or decrease all weights in a single step.

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$




What can we say about the gradients on **w**?

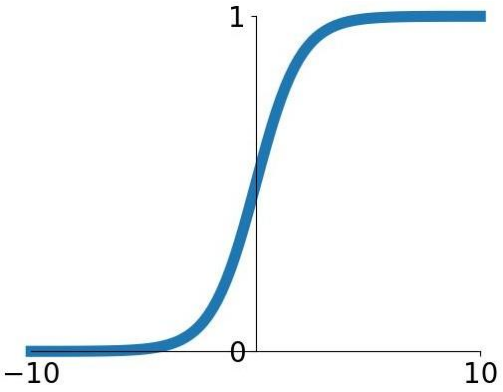
Always all positive or all negative :(

How to improve?

**For a single element! Minibatches help**

# Activation Functions

$$\sigma(x) = 1/(1 + e^{-x})$$



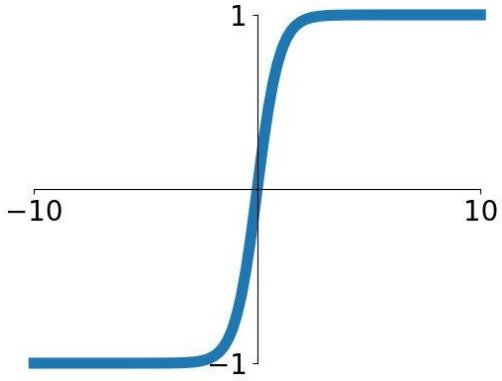

**Sigmoid**

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

1. Saturated neurons “kill” the gradients
2. outputs are not zero-centered



# Activation Functions



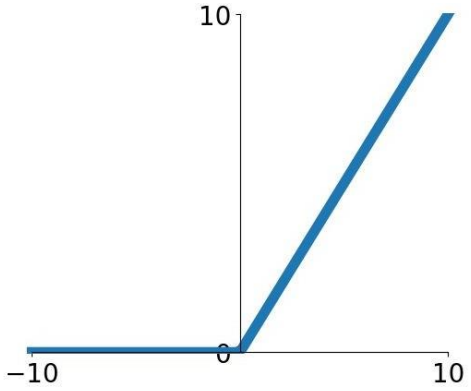

**tanh(x)**

- Squashes numbers to range [-1,1]
- zero centered (nice)
- still kills gradients when saturated :(

[LeCun et al., 1991]



# Activation Functions




**ReLU**
(Rectified Linear Unit)

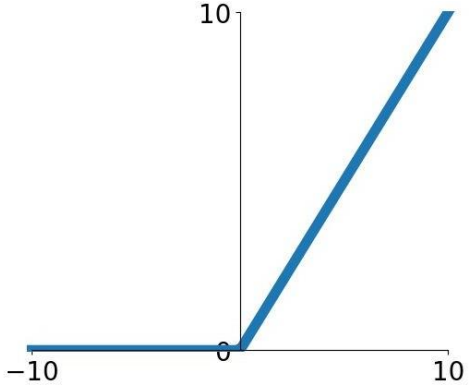
- Computes **f(x) = max(0,x)**

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

[Krizhevsky et al., 2012]



# Activation Functions




**ReLU**
(Rectified Linear Unit)

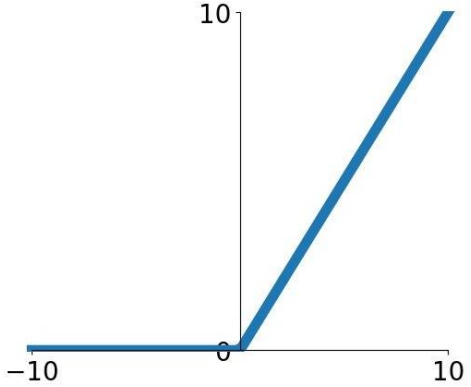
- Computes **f(x) = max(0,x)**

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

- Not zero-centered output



# Activation Functions

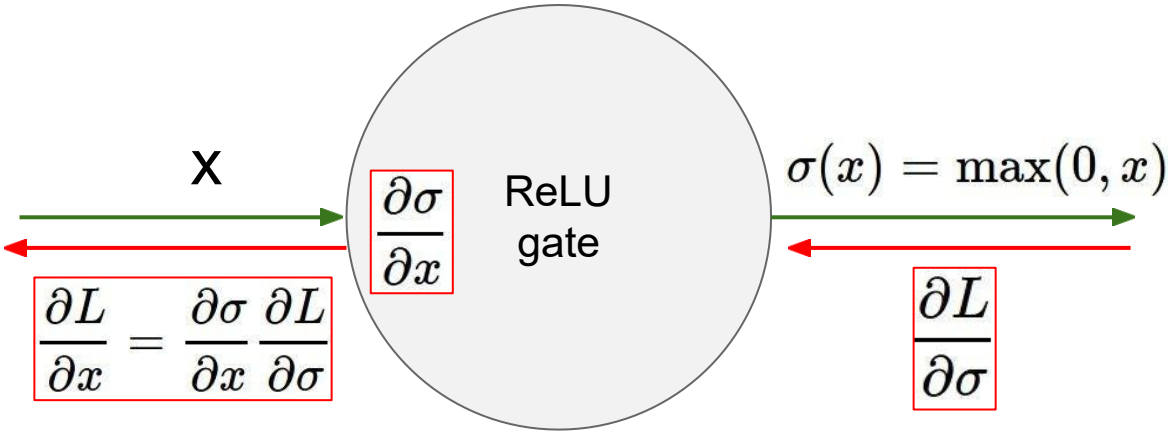


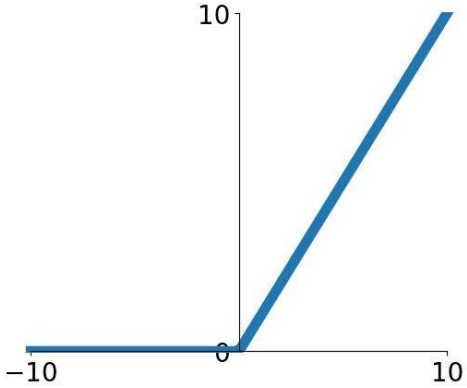

**ReLU**
(Rectified Linear Unit)

- Computes **f(x) = max(0,x)**
- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)

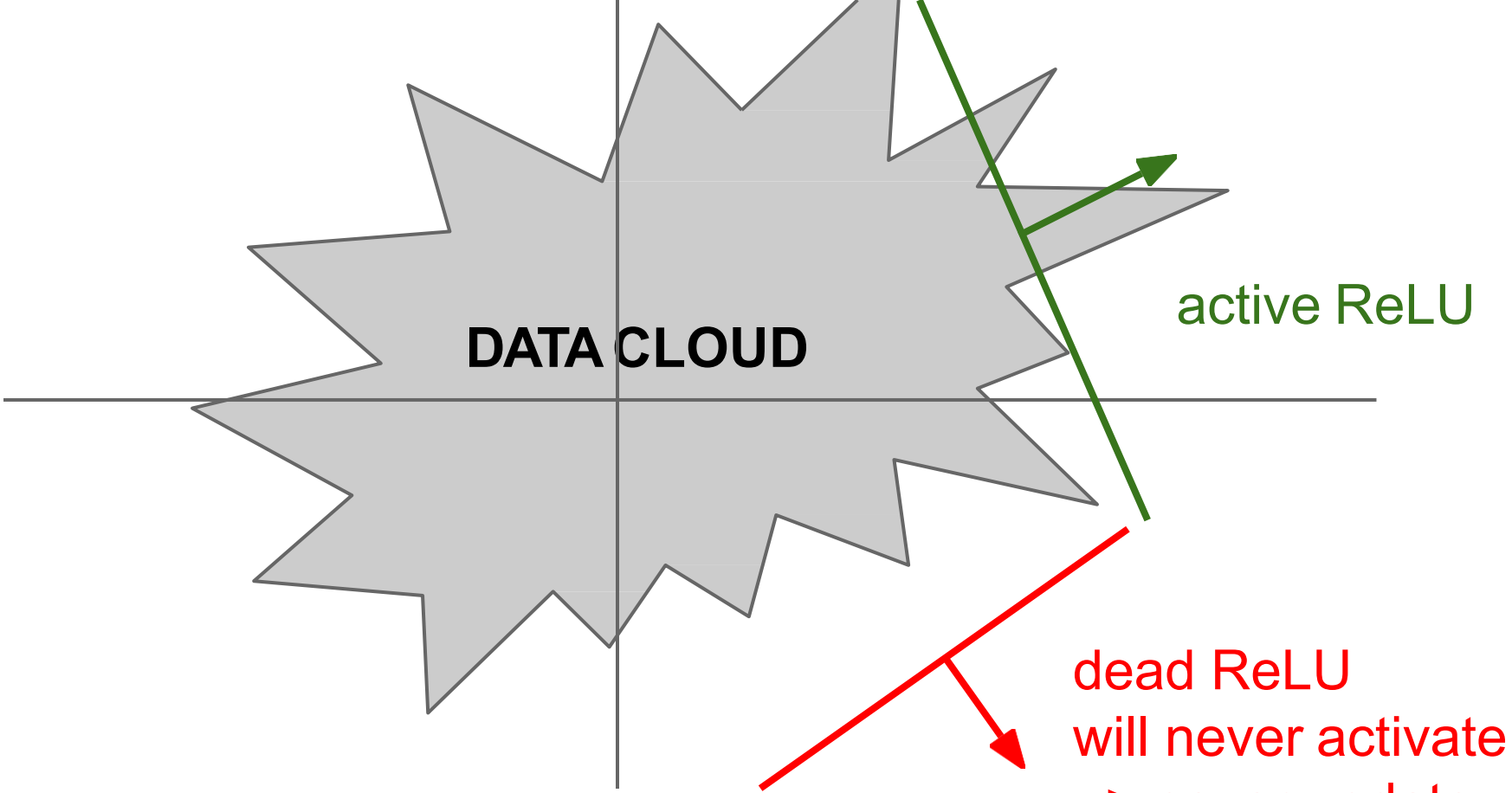
- Not zero-centered output
- An annoyance: dead neuron

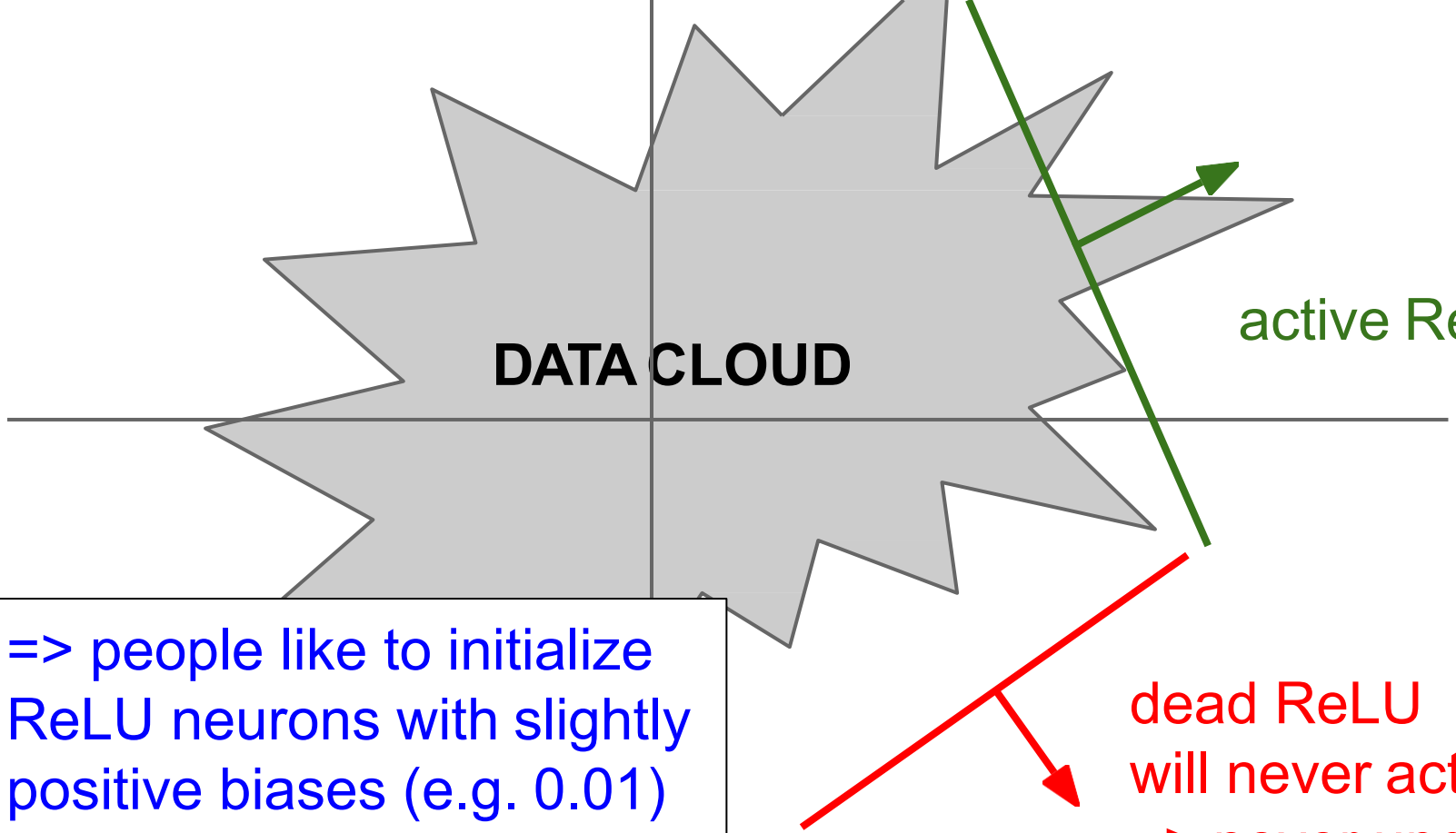




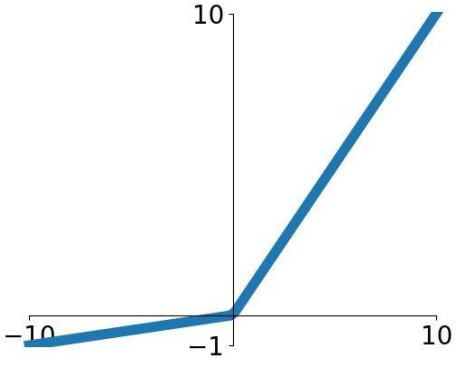

What happens when x = -10?
What happens when x = 0?
What happens when x = 10?


DATA CLOUD
active ReLU
dead ReLU
will never activate
=> never update




=> people like to initialize ReLU neurons with slightly positive biases (e.g. 0.01)

# Activation Functions



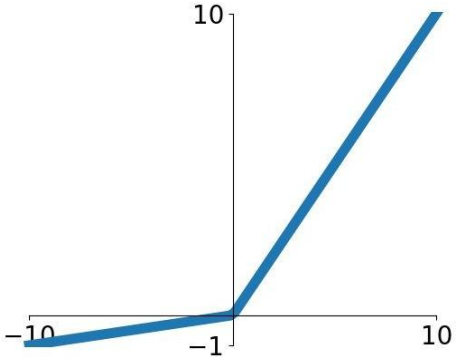

**Leaky ReLU**

$$f(x) = \max(0.01x, x)$$

- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not “die”.**



# Activation Functions




- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not “die”.**

**Leaky ReLU**

$$f(x) = \max(0.01x, x)$$

**Parametric Rectifier (PReLU)**

$$f(x) = \max(\alpha x, x)$$

backprop into \alpha (parameter)

# Maxout “Neuron”

- Does not have the basic form of dot product -> element-wise nonlinearity
- Generalizes ReLU and Leaky ReLU
- Linear Regime. Does not saturate. Does not die.

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

Problem: doubles the number of parameters/neuron :(



# In practice:

- Use ReLU. Be careful with your learning rates
- Try out Leaky ReLU / Maxout to squeeze out some marginal gains
- Don’t use sigmoid or tanh in hidden layers
  - they can be used in output layers to control the output range



# Convolution



# Previous: Linear model for image classification



$I \in R^{32\times32\times3}$



$\boldsymbol{x} \in R^{3072\times1}$

vectorization

$f(\boldsymbol{x}, \boldsymbol{W}, \boldsymbol{b})$

$\boldsymbol{W}, \boldsymbol{b}$

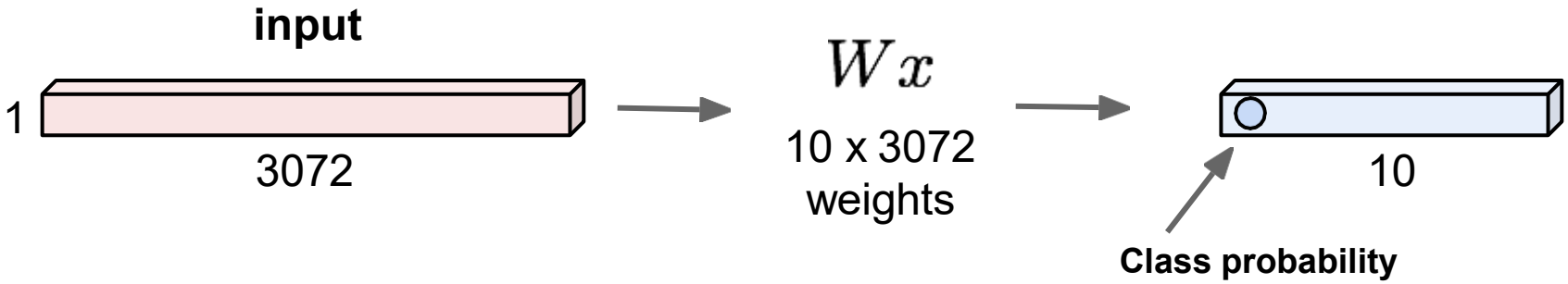
parameters

**10** numbers giving class scores

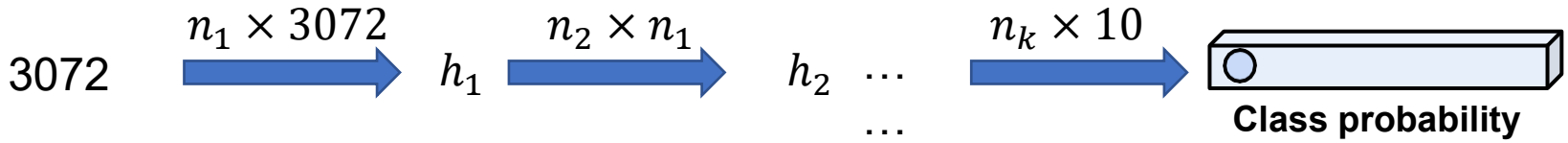
Prediction: largest score



# Previous: Linear model for image classification

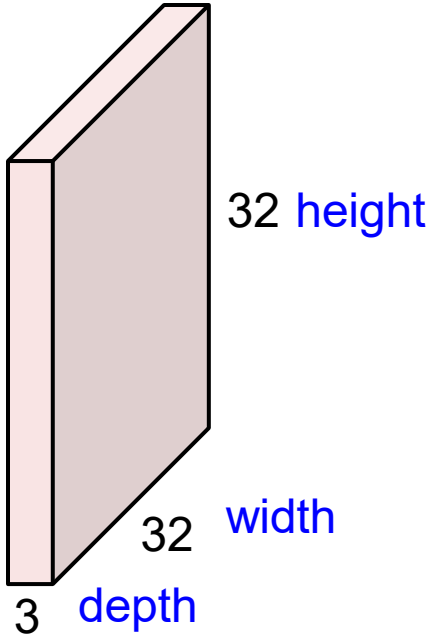



Stacking many layers to form the multi-layer perception:

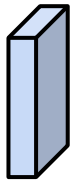



The model needs to learn a large number of parameters

# Convolutional layers




5x5x3 filter

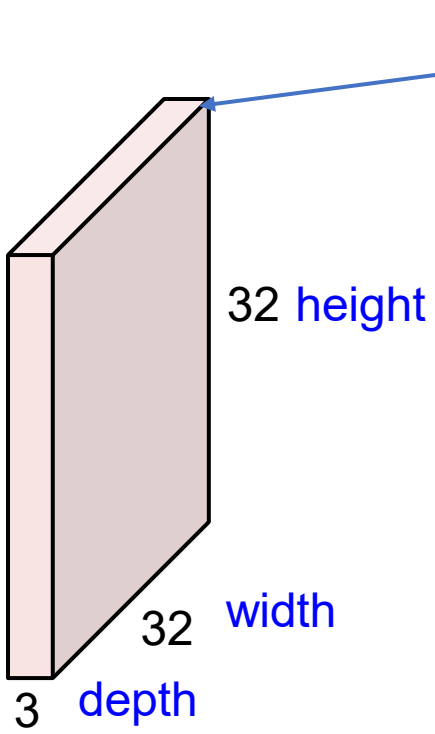


**Convolve** the filter with the image
i.e. “slide over the image spatially, computing dot products"

Number of parameters:

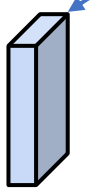
$$5 \times 5 \times 3 = 45$$

# Convolutional layers




Filters always extend the full depth of the input volume

5x5x3 filter

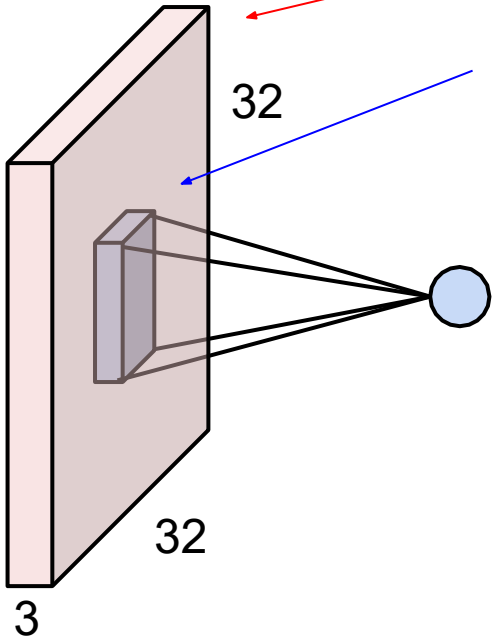


**Convolve** the filter with the image
i.e. “slide over the image spatially, computing dot products

Number of parameters:

$$5 \times 5 \times 3 = 45$$

# Convolution Layer



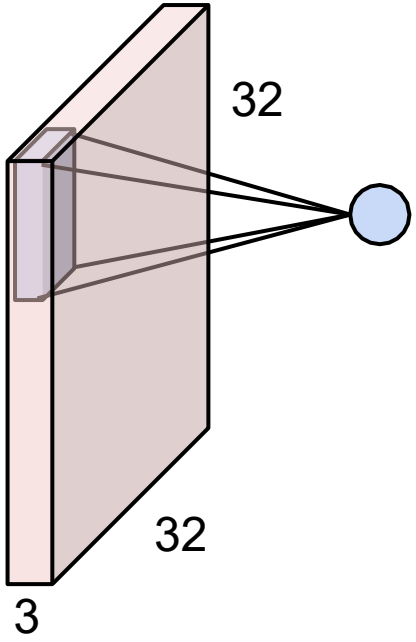

32x32x3 image

5x5x3 filter $w$

**1 number:**
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

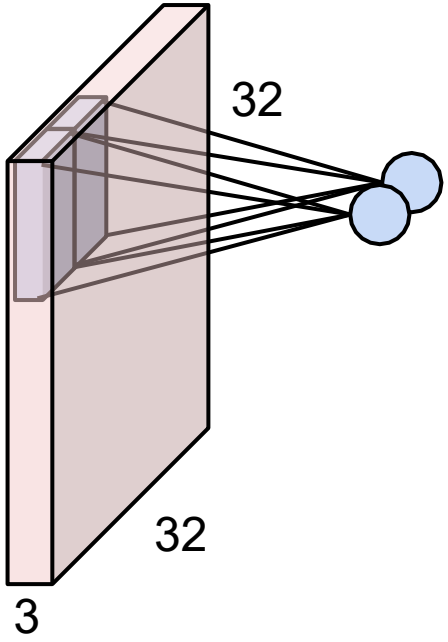
$$w^T x + b$$

# Convolution Layer



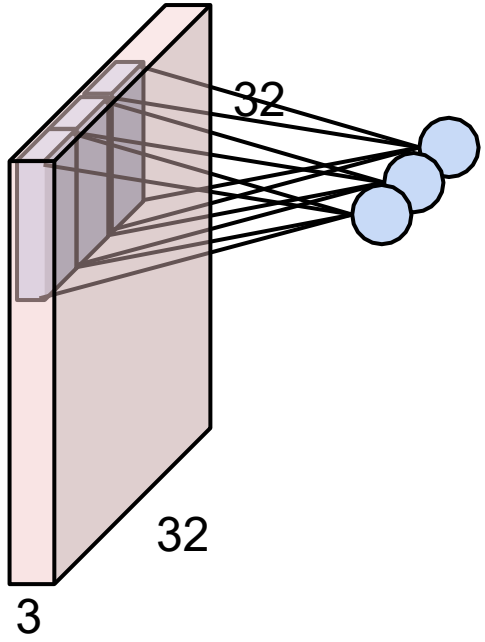



# Convolution Layer



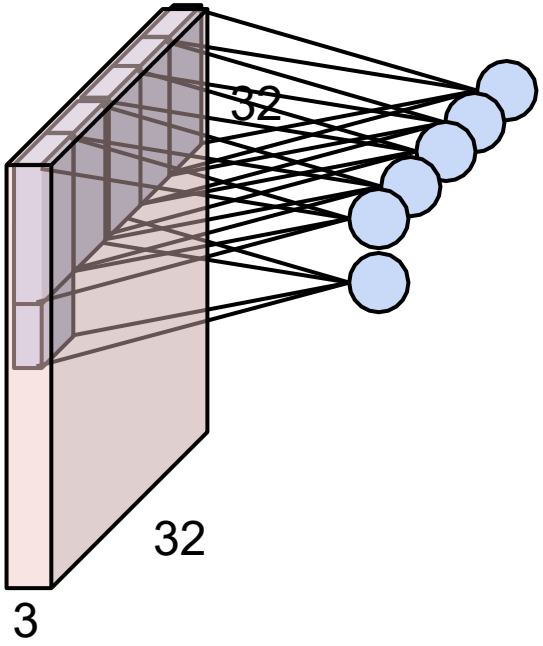



# Convolution Layer



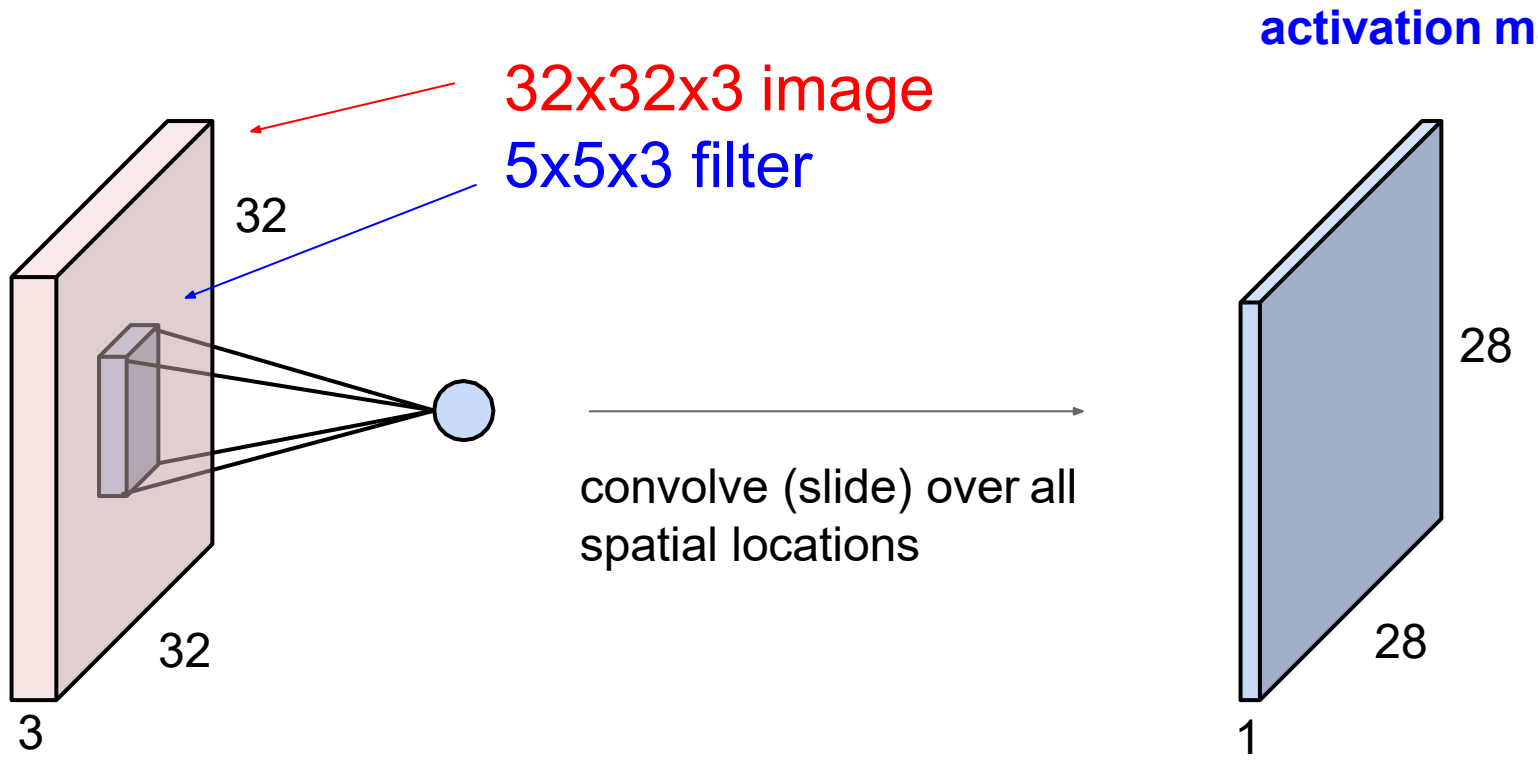



# Convolution Layer






# Convolution Layer

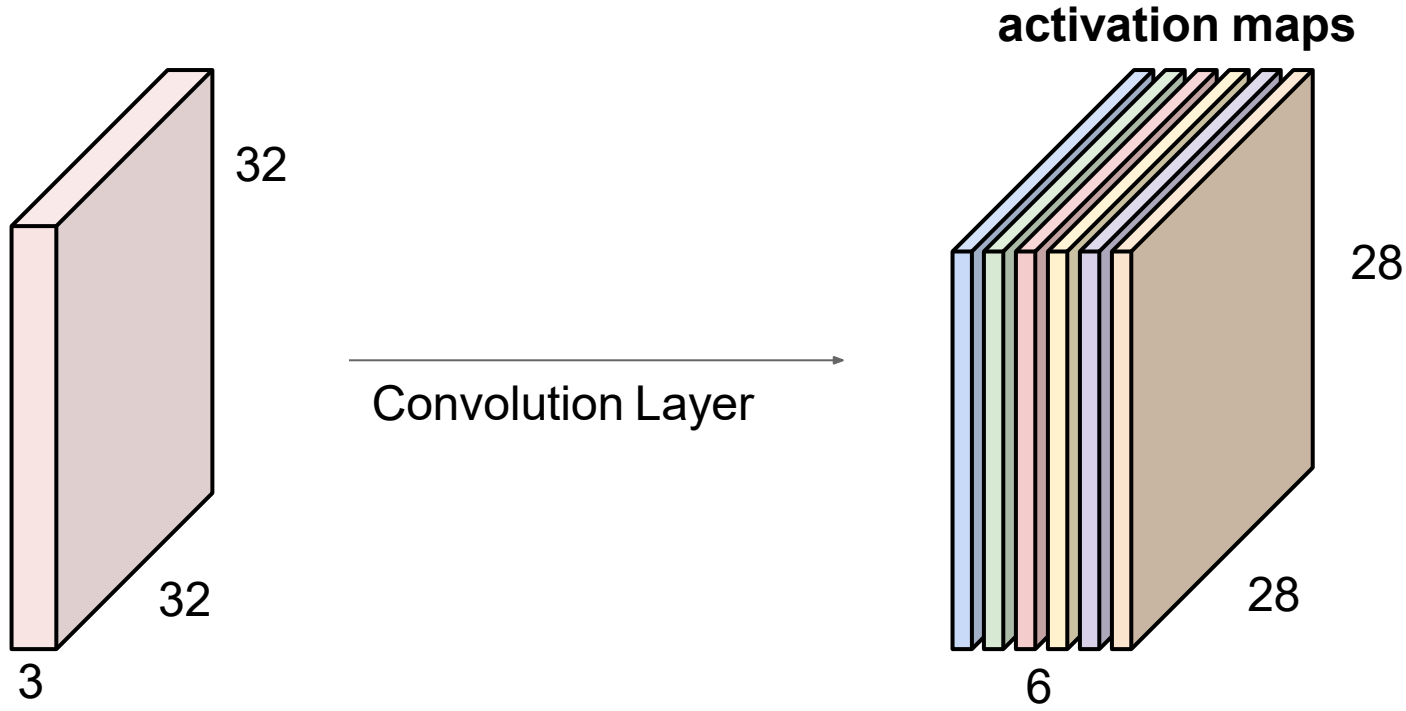


# Convolution Layer

consider a **second**, green filter



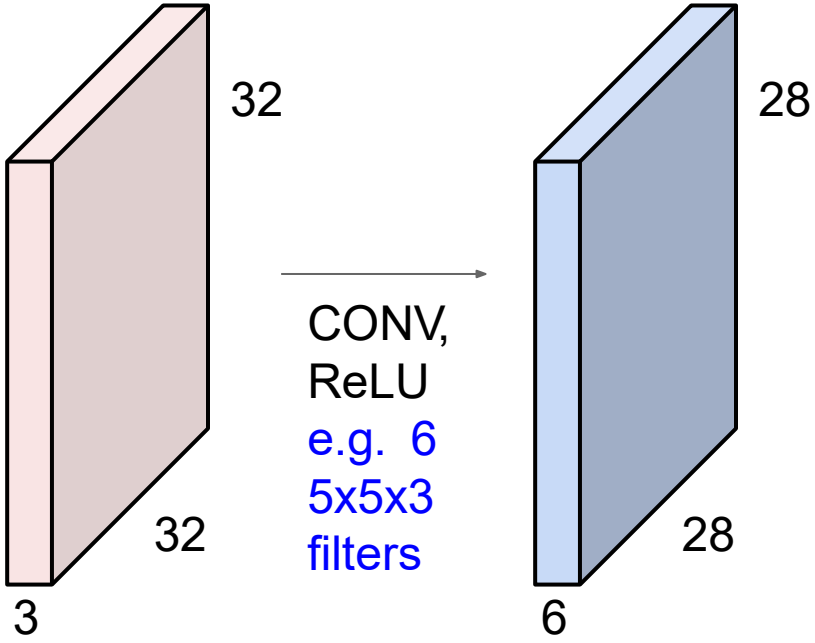
For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:




We stack these up to get a "new image" of size 28x28x6!

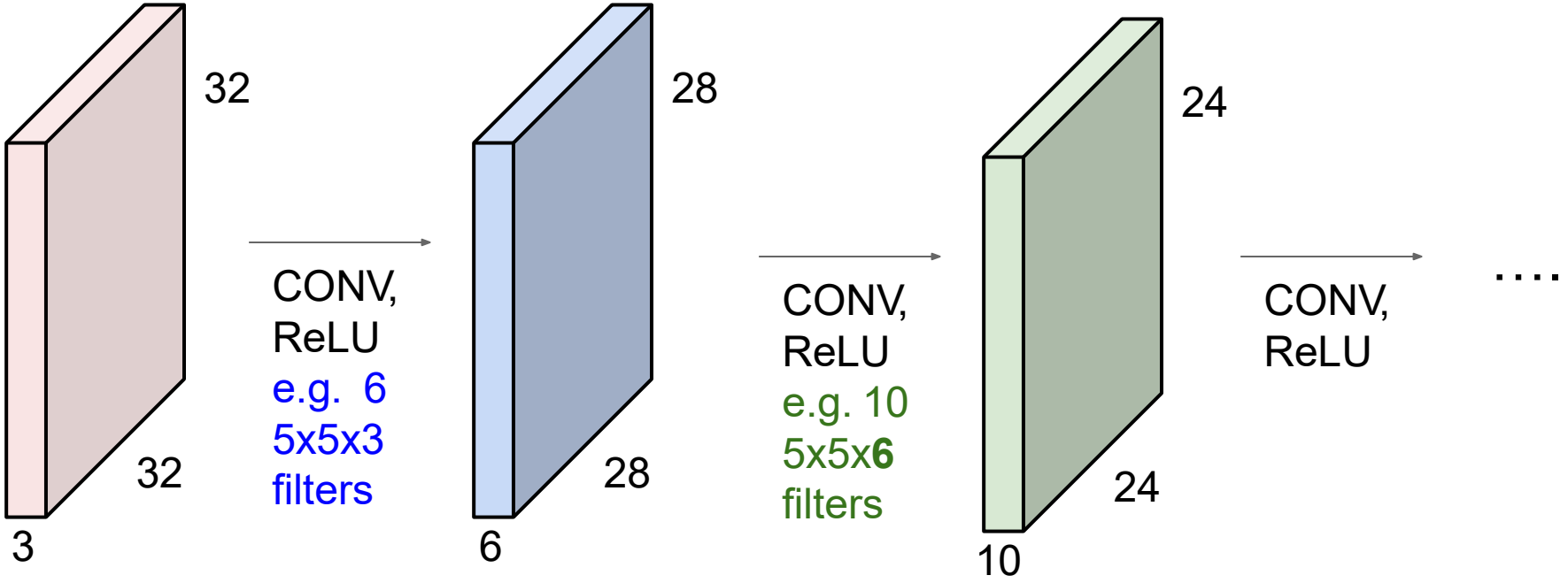


**Preview:** ConvNet is a sequence of Convolution Layers, interspersed with activation functions

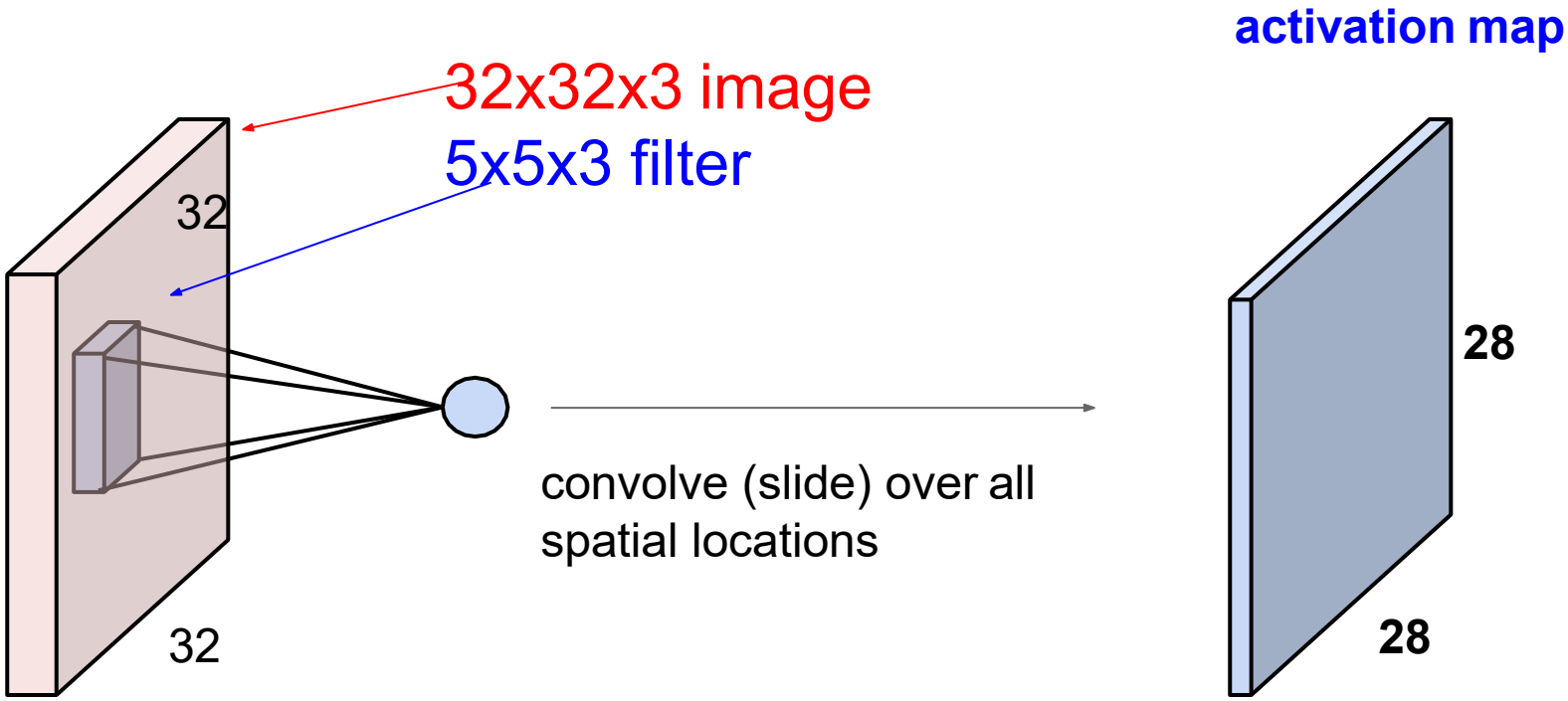




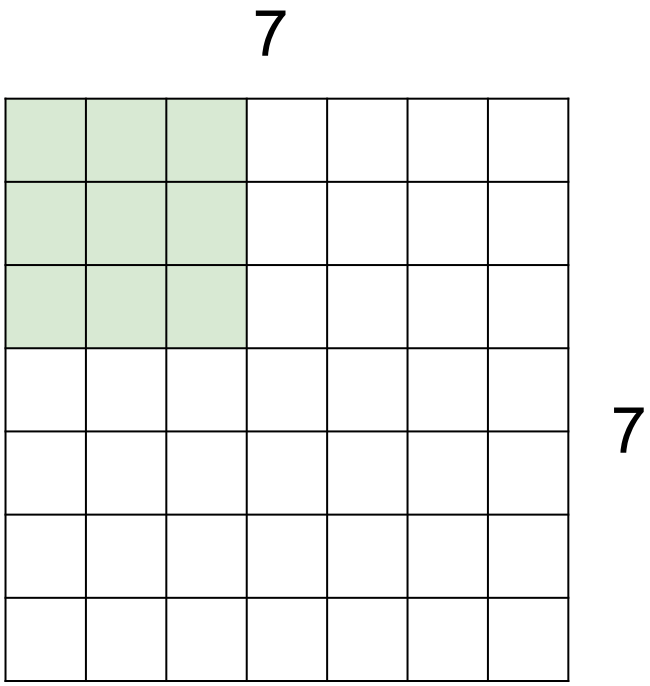

ConvNet is a sequence of Convolution Layers, interspersed with activation functions



# A closer look at spatial dimensions:



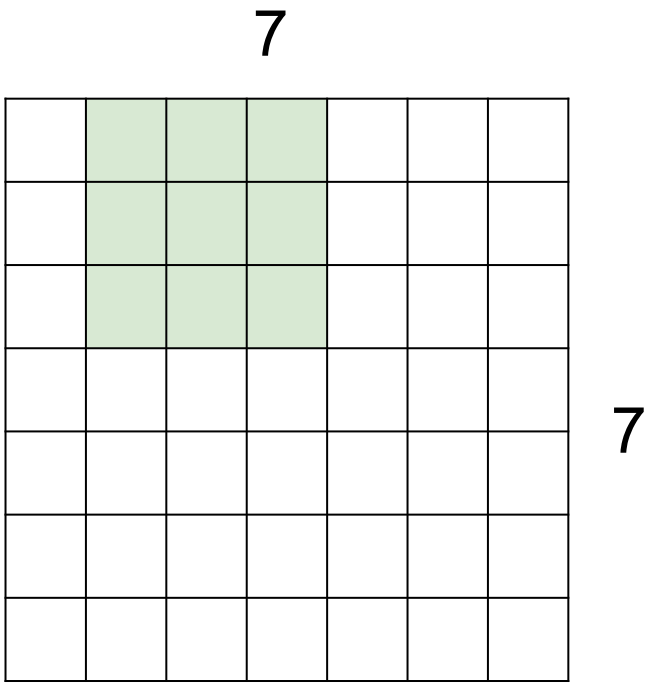
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter



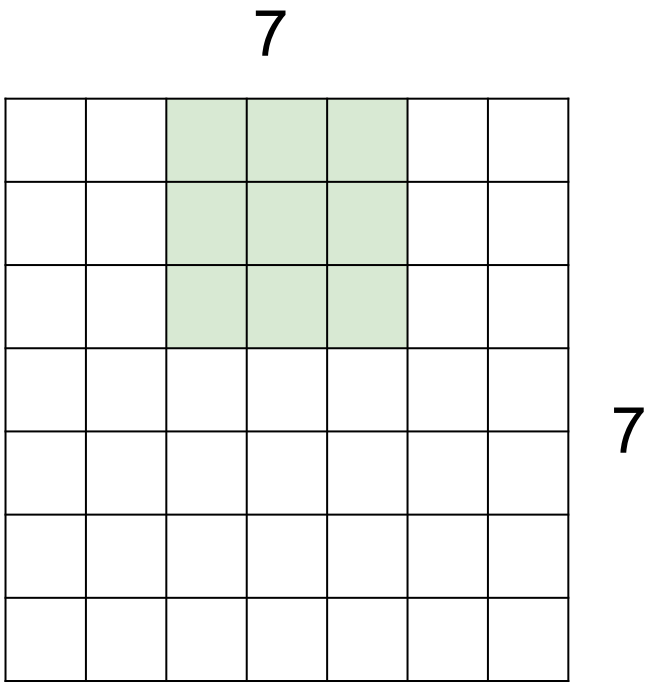
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter



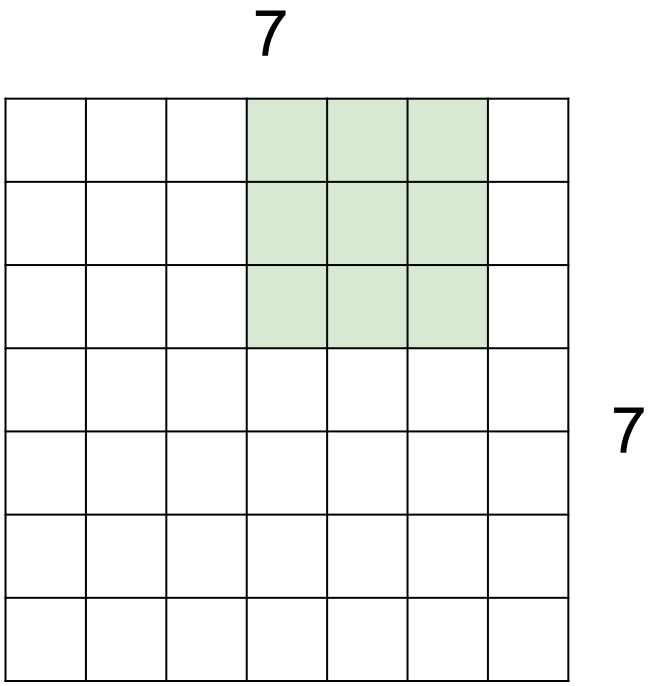
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter



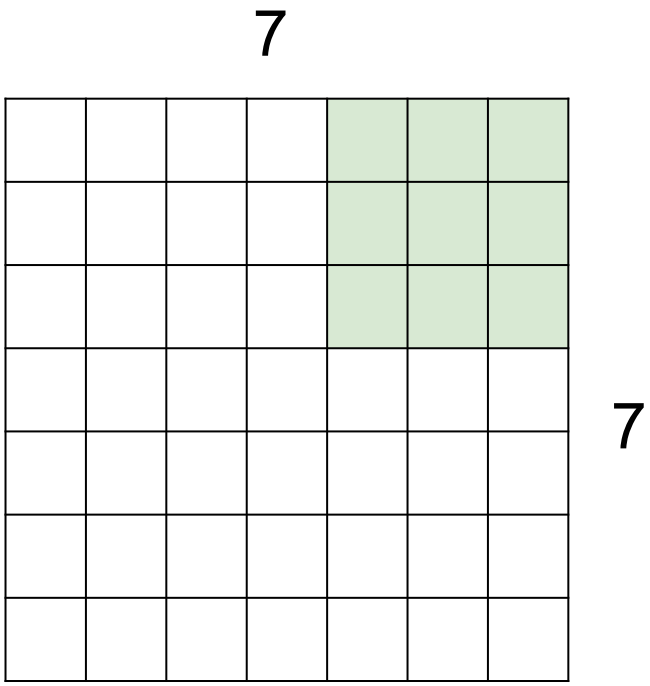
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter



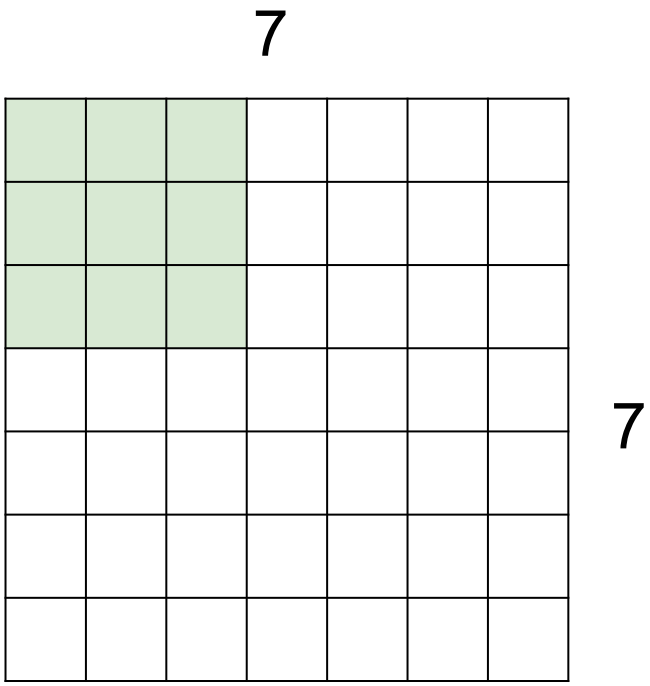
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter

**=> 5x5 output**

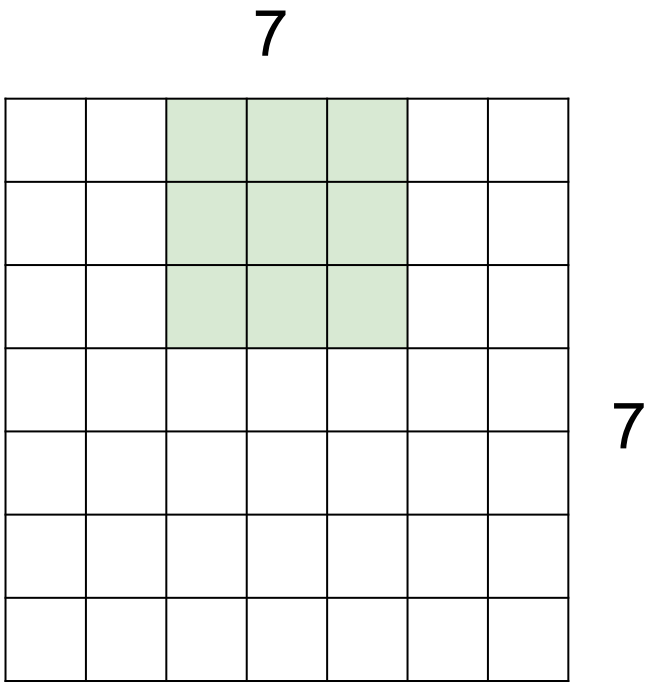
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**



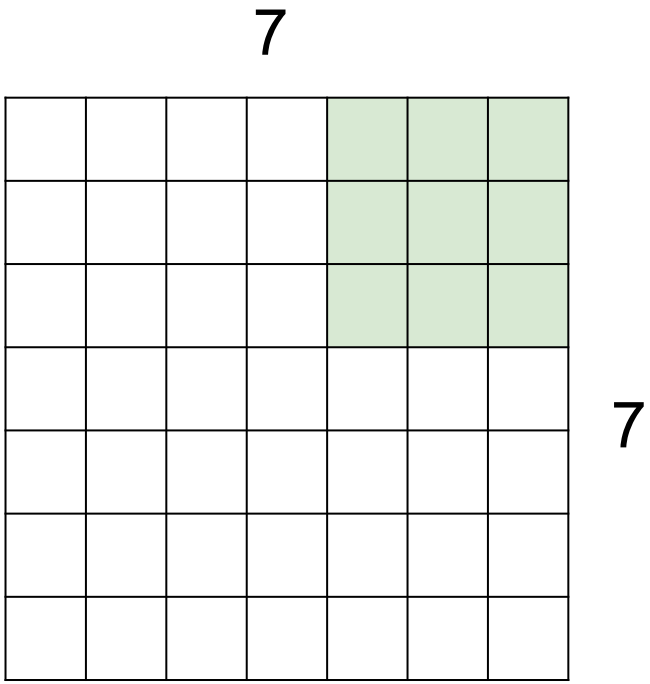
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**



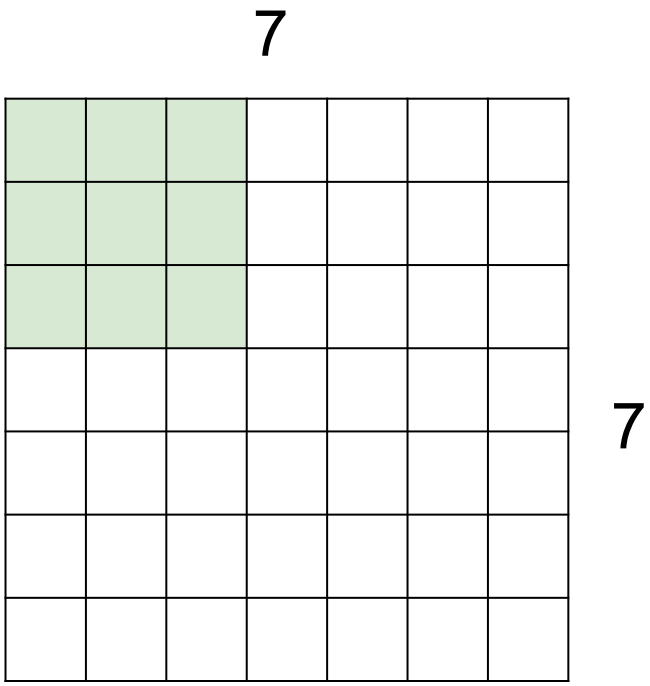
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
**=> 3x3 output!**



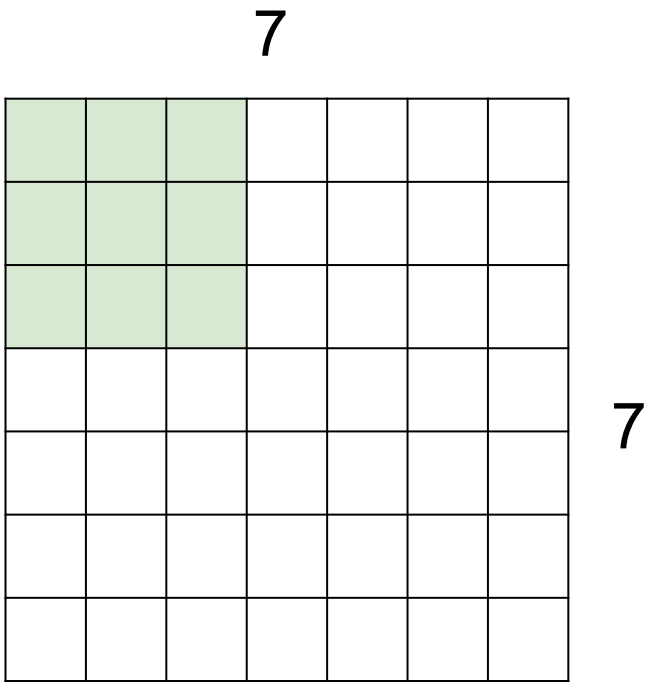
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**



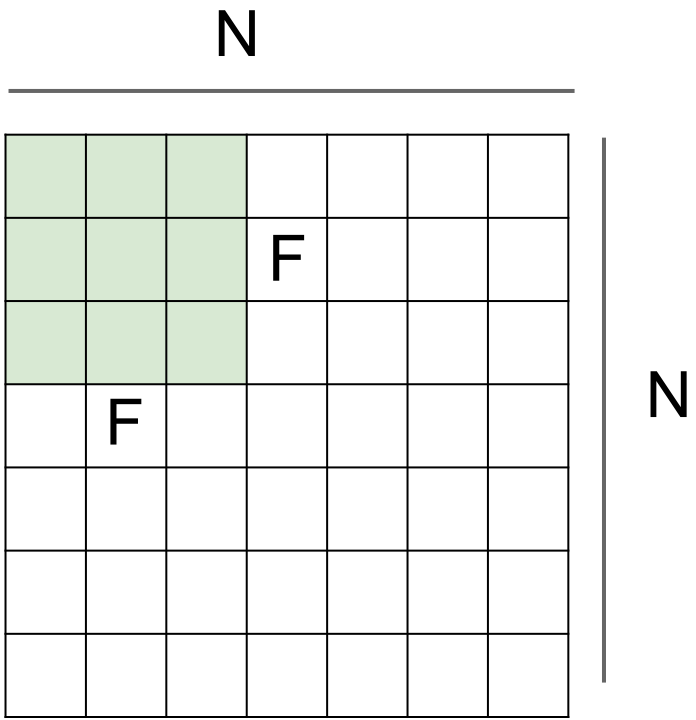
# A closer look at spatial dimensions:




7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

**doesn't fit!**
cannot apply 3x3 filter on
7x7 input with stride 3.



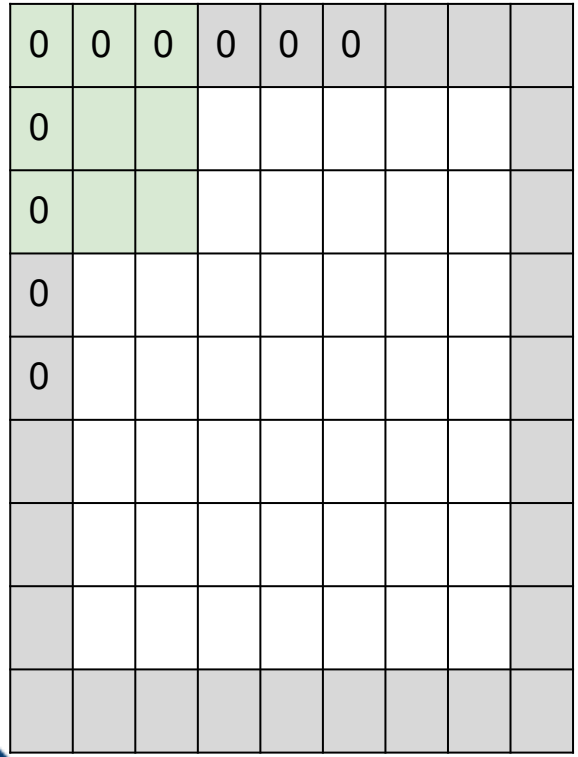



Output size:
**(N - F) / stride + 1**

e.g. N = 7, F = 3:
stride 1 => (7 - 3)/1 + 1 = 5
stride 2 => (7 - 3)/2 + 1 = 3
stride 3 => (7 - 3)/3 + 1 = 2.33



# In practice: Common to zero pad the border

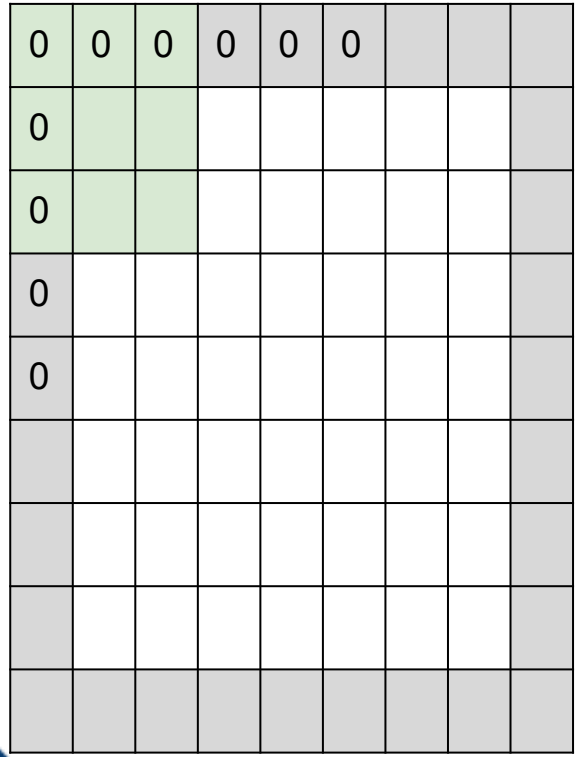



e.g. input 7x7
**3x3** filter, applied with **stride 1**
**pad with 1 pixel** border => what is the output?

(recall:)
(N - F) / stride + 1



# In practice: Common to zero pad the border

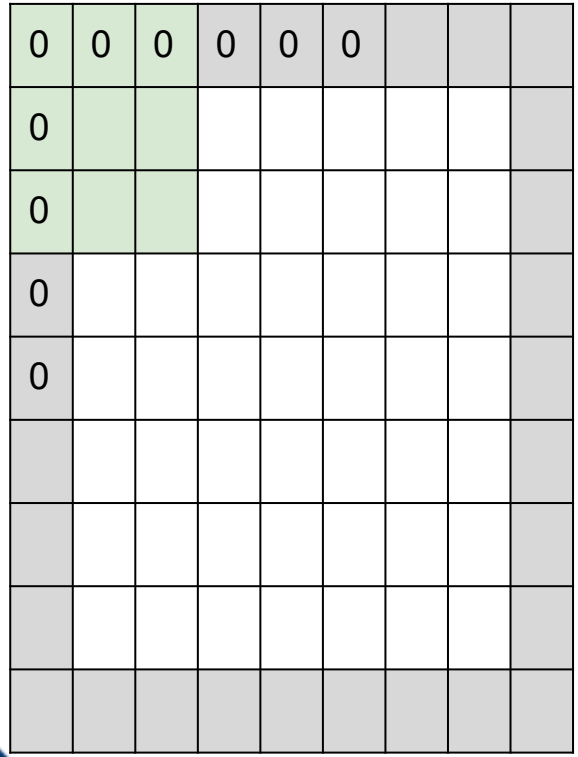



e.g. input 7x7
**3x3** filter, applied with **stride 1**
**pad with 1 pixel** border => what is the output?

**7x7 output!**

(recall:)
(N + 2P - F) / stride + 1

# In practice: Common to zero pad the border



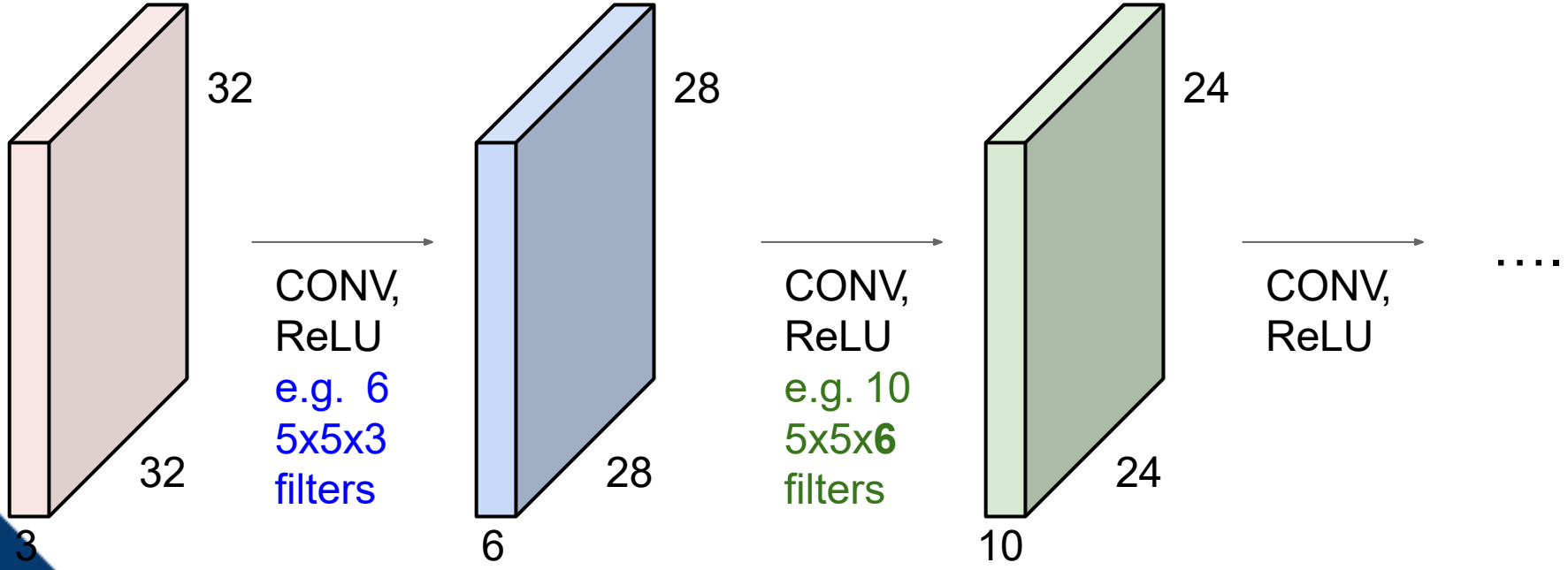

e.g. input 7x7
**3x3** filter, applied with **stride 1**
**pad with 1 pixel** border => what is the output?

**7x7 output!**
in general, common to see CONV layers with stride 1, filters of size FxF, and zero-padding with (F-1)/2. (will preserve size spatially)
e.g. F = 3 => zero pad with 1
F = 5 => zero pad with 2
F = 7 => zero pad with 3

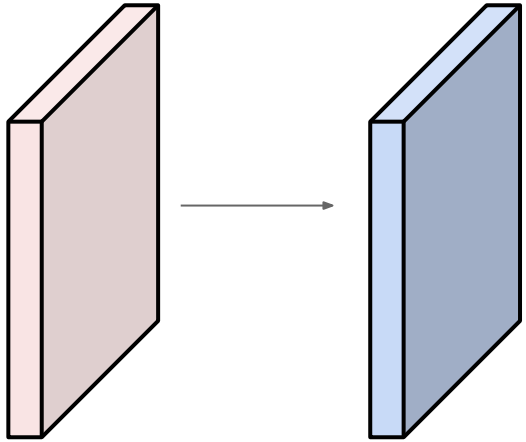


**Remember back to…**

E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 -> 28 -> 24 ...). Shrinking too fast is not good, doesn't work well.



Examples time:

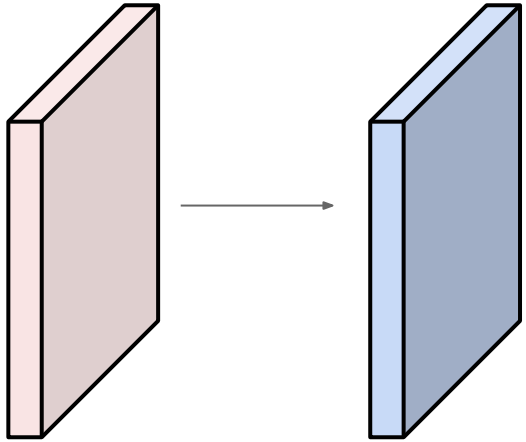


Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size: ?



Examples time:

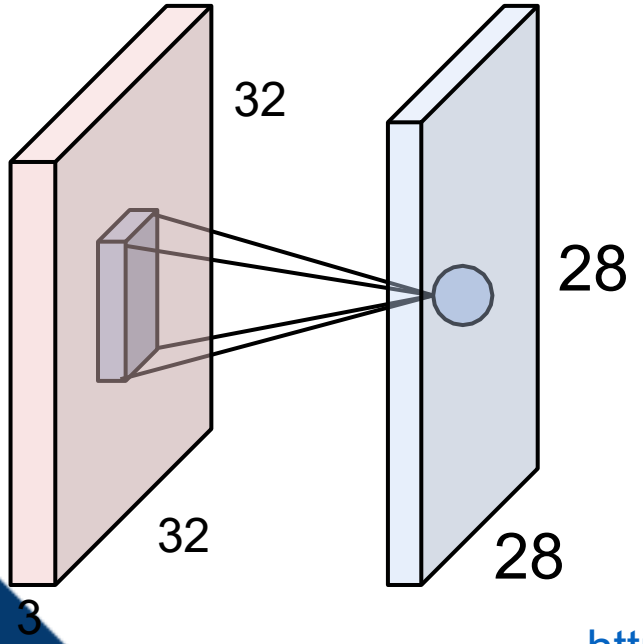


Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Number of parameters in this layer?



# Receptive field



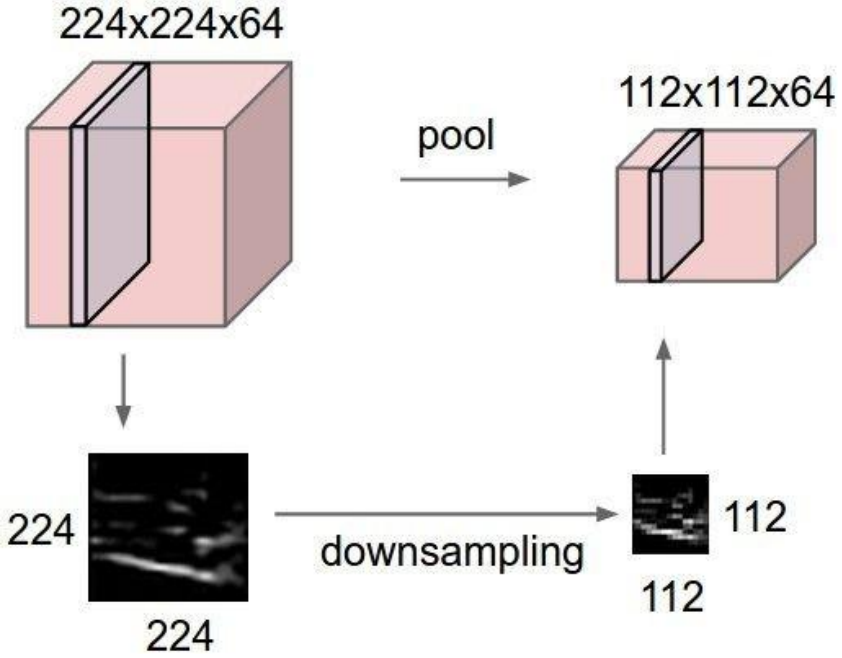

An activation map is a 28x28 sheet of neuron outputs:

1. Each is connected to a small region in the input
2. All of them share parameters

"5x5 filter" -> "5x5 receptive field for each neuron"

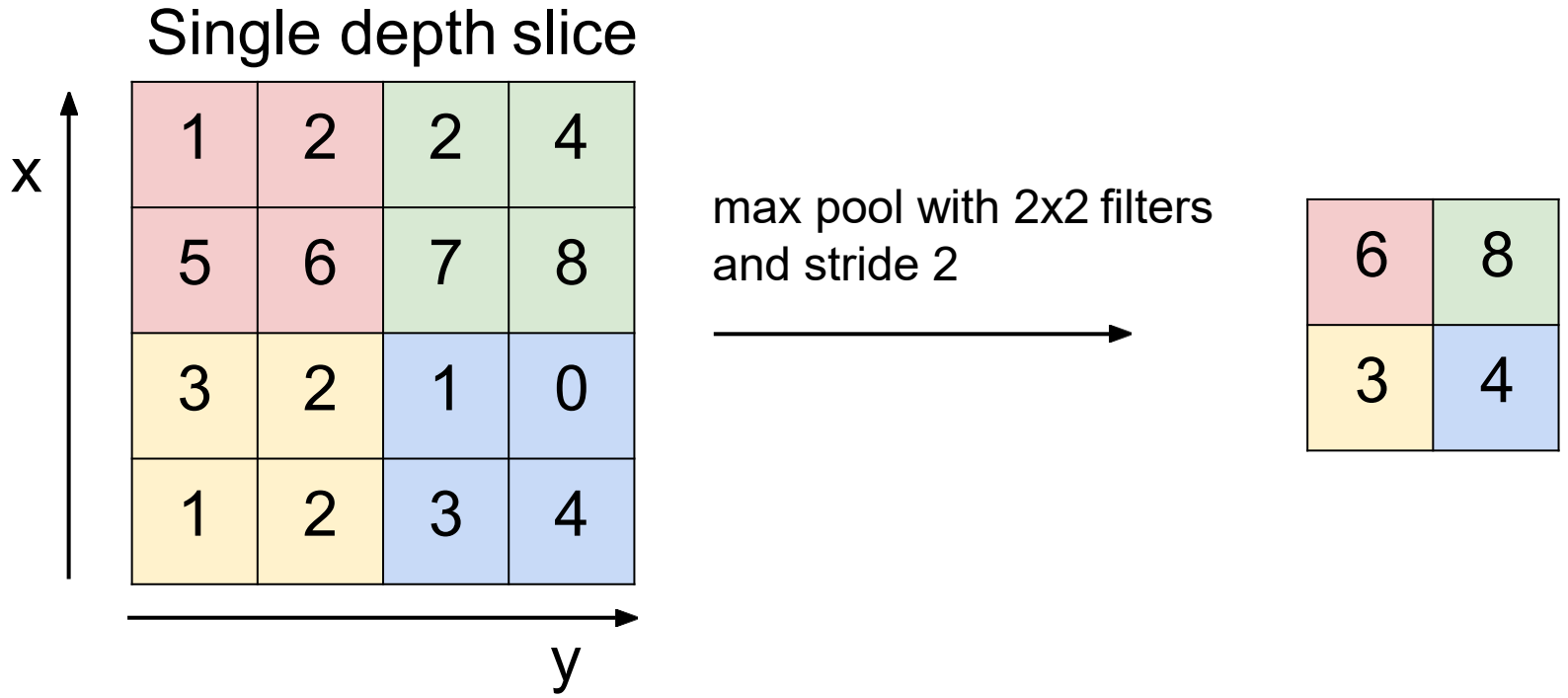
https://distill.pub/2019/computing-receptive-fields/

# Pooling layer




- makes the representations smaller and more manageable
- operates over each activation map independently:

# Max pooling


Single depth slice
x
y
1 2 2 4
5 6 7 8
3 2 1 0
1 2 3 4
max pool with 2x2 filters
and stride 2
6 8
3 4



**Anonymous**
**Class Feedback**

# Q&A